

HealthPulse- Real time AI based Health Trend Monitoring &  
Risk Prediction System  
21CSP302L- PROJECT

*Submitted by*  
ADITYA JHA [RA2311028010013]  
KSHITIJ TIWARI [RA2311028010018]

*Under the Guidance of*  
Dr. STEPHEN DASS A  
Assistant Professor, Department of Networking and Communications

*in partial fulfillment of the requirements for the degree of*  
BACHELOR OF TECHNOLOGY  
in  
COMPUTER SCIENCE ENGINEERING  
with specialization in Cloud Computing



DEPARTMENT OF NETWORKING AND COMMUNICATIONS  
COLLEGE OF ENGINEERING AND TECHNOLOGY  
SRM INSTITUTE OF SCIENCE AND TECHNOLOGY  
KATTANKULATHUR- 603 203

MAY 2026



Department of Networking and Communications  
**SRM Institute of Science & Technology**  
**Own Work Declaration Form**

This sheet must be filled in (each box ticked to show that the condition has been met). It must be signed and dated along with your student registration number and included with all assignments you submit – work will not be marked unless this is done.

To be completed by the student for all assessments

**Degree/ Course** : B. Tech in CSE w/s Cloud Computing

**Student Name** : Aditya Jha, Kshitij Tiwari

**Registration Number** : RA2311028010013, RA2311028010018

**Title of Work** : HealthPulse- Real time AI based Health Trend Monitoring & Risk Prediction System

I / We hereby certify that this assessment compiles with the University's Rules and Regulations relating to Academic misconduct and plagiarism, as listed in the University Website, Regulations, and the Education Committee guidelines.

I / We confirm that all the work contained in this assessment is my / our own except where indicated, and that I / We have met the following conditions:

- Clearly referenced / listed all sources as appropriate
- Referenced and put in inverted commas all quoted text (from books, web, etc)
- Given the sources of all pictures, data etc. that are not my own
- Not made any use of the report(s) or essay(s) of any other student(s) either past or present
- Acknowledged in appropriate places any help that I have received from others (e.g. fellow students, technicians, statisticians, external sources)
- Compiled with any other plagiarism criteria specified in the Course handbook /University website

I understand that any false claim for this work will be penalized in accordance with the University policies and regulations.

**DECLARATION:**

I am aware of and understand the University's policy on Academic misconduct and plagiarism and I certify that this assessment is my / our own work, except where indicated by referring, and that I have followed the good academic practices noted above.

ADITYA JHA

KSHITIJ TIWARI

If you are working in a group, please write your registration numbers and sign with the date for every student in your group.



# SRM INSTITUTE OF SCIENCE AND TECHNOLOGY KATTANKULATHUR – 603 203

## BONAFIDE CERTIFICATE

Certified that 21CSP302L - Project report titled “HEALTHPULSE – REAL TIME AI BASED HEALTH TREND MONITORING & RISK PREDICTION SYSTEM” is the bonafide work of “ADITYA JHA [RA2311028010013], KSHITIJ TIWARI [RA2311028010018]” who carried out the project work under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

### **SIGNATURE**

Dr. Stephen Dass A

### **SUPERVISOR**

Professor  
DEPARTMENT OF  
NETWORKING &  
COMMUNICATIONS

### **SIGNATURE**

DR. M. Laksmi

### **PROFESSOR & HEAD**

DEPARTMENT OF  
NETWORKING &  
COMMUNICATIONS

### **EXAMINER 1**

**Name & Signature**

### **EXAMINER 2**

**Name & Signature**

## ACKNOWLEDGEMENTS

We express our humble gratitude to **Dr. C. Muthamizhchelvan**, Vice-Chancellor, SRM Institute of Science and Technology, for the facilities extended for the project work and his continued support.

We extend our sincere thanks to **Dr. Leenus Jesu Martin M**, Dean-CET, SRM Institute of Science and Technology, for his invaluable support.

We wish to thank **Dr. Revathi Venkataraman**, Professor and Chairperson, School of Computing, SRM Institute of Science and Technology, for her support throughout the project work.

We encompass our sincere thanks to, **Dr. M. Pushpalatha**, Professor and Associate Chairperson - CS, School of Computing and **Dr.C. Lakshmi**, Professor and Associate Chairperson -AI, School of Computing, SRM Institute of Science and Technology, for their invaluable support.

We are incredibly grateful to our Head of the Department, **Dr. M. Lakshmi**, Professor & Head Department of Networking & Communications, SRM Institute of Science and Technology, for her suggestions and encouragement at all the stages of the project work.

We want to convey our thanks to our Project Coordinators, Panel Head, and Panel Members Department of Computational Intelligence, SRM Institute of Science and Technology, for their inputs during the project reviews and support.

We register our immeasurable thanks to our Faculty Advisor, **Dr. N. Senthamarai**, Department of Networking & Communications, SRM Institute of Science and Technology, for leading and helping us to complete our course.

Our inexpressible respect and thanks to our guide, **Dr. Stephen Dass A**, Department of Networking & Communications, SRM Institute of Science and Technology, for providing us with an opportunity to pursue our project under his mentorship. He provided us with the freedom and support to explore the research topics of our interest. His passion for solving problems and making a difference in the world has always been inspiring.

We sincerely thank all the staff members of Department of Networking & Communications, School of Computing, S.R.M Institute of Science and Technology, for their help during our project. Finally, we would like to thank our parents, family members, and friends for their unconditional love, constant support and encouragement

Authors

# ABSTRACT

Timely identification of disease outbreaks is one of the most serious problems because of delays in the conventional surveillance framework, piecemeal information data, and the increasing power of fake news. In this study, we describe a data-driven framework named HealthPulse that is aimed at providing near real-time disease trend analysis, based on publicly available behavioral and environmental data. The system combines multi-source inputs, such as the search data provided by Google Trends, weather parameters and filtered news signals, to determine the emergence of health risks without using limited clinical data.

The suggested solution uses a data pipeline that will be structured to automate the process of data collection, preprocessing and storage based on cloud-based infrastructure. The data collected are normalized, cleaned, and engineered to feature to maintain consistency among heterogeneous sources. It has a multi-layer validation system to reduce the effects of fake news, anomalous spikes, and manipulated trends by verifying cross-source, detecting statistical anomalies, and scoring credibility.

To predictively analyze, the system employs a hybrid machine learning approach. A time-series model based on SARIMAX is applied to identify both the temporal and seasonal trends in the disease-related search behavior, and an XGBoost classifier is applied to identify risk levels of outbreaks in preset severity levels. The models are trained using structured data based on historical trends and engineered data like growth rates, rolling averages, and anomaly indicators. Experimental analysis shows that the forecasting model has consistent error margin that can be used to make short term forecasts whereas the classification model has high accuracy and consistency among validation folds.

Scalability and reproducibility are built into the system with modular architecture of the backend, and automated workflows of data ingestion. Early findings show that behavioral search data, together with environmental factors, and confirmed by multi-layer filtering can give early clues of impending outbreaks a few days before conventional reporting systems.

This paper underscores the possibility of inexpensive and accessible, and scalable disease surveillance systems that do not rely on specialized equipment or other confidential medical information. The next round of improvements will be on adding more sources of real time data, enhancing the robustness of the anomaly detection and extending the framework to multi- region deployment to make it more applicable.

# TABLE OF CONTENTS

|                          |                                                                   |                 |
|--------------------------|-------------------------------------------------------------------|-----------------|
| <b>ABSTRACT</b>          |                                                                   | <b>v</b>        |
| <b>TABLE OF CONTENTS</b> |                                                                   | <b>vi</b>       |
| <b>LIST OF FIGURES</b>   |                                                                   | <b>viii</b>     |
| <b>LIST OF TABLES</b>    |                                                                   | <b>ix</b>       |
| <b>ABBREVIATIONS</b>     |                                                                   | <b>x</b>        |
| <b>CHAPTER NO.</b>       | <b>TITLE</b>                                                      | <b>PAGE NO.</b> |
| <b>1</b>                 | <b>INTRODUCTION</b>                                               | <b>1</b>        |
|                          | 1.1 Introduction to Project                                       | 1               |
|                          | 1.2 Problem Statement and Description                             | 2               |
|                          | 1.3 Motivation                                                    | 2               |
|                          | 1.4 Sustainable Development Goal of the Project                   | 3               |
| <b>2</b>                 | <b>LITERATURE SURVEY</b>                                          | <b>5</b>        |
|                          | 2.1 Overview of the Research Area                                 | 5               |
|                          | 2.2 Existing Models and Frameworks                                | 6               |
|                          | 2.3 Limitations Identified from Literature Survey (Research Gaps) | 7               |
|                          | 2.4 Research Objectives                                           | 8               |
|                          | 2.5 Product Backlog (Key user stories with Desired outcomes)      | 9               |
|                          | 2.6 Plan of Action (Project Road Map)                             | 10              |
| <b>3</b>                 | <b>SPRINT PLANNING AND EXECUTION METHODOLOGY</b>                  | <b>12</b>       |
|                          | 3.1 SPRINT I -Data Engineering & Infrastructure Setup             | 12              |
|                          | 3.1.1 Objectives with user stories of Sprint I                    | 12              |
|                          | 3.1.2 Functional Document                                         | 12              |
|                          | 3.1.3 Architecture Document                                       | 16              |
|                          | 3.1.4 Outcome of objectives/ Result Analysis                      | 16              |
|                          | 3.1.5 Sprint Retrospective                                        | 17              |
|                          | 3.2 SPRINT II Advanced Validation/Intelligence Layer              | 18              |
|                          | 3.2.1 Objectives with user stories of Sprint II                   | 18              |
|                          | 3.2.2 Functional Document                                         | 18              |
|                          | 3.2.3 Architecture Document                                       | 19              |
|                          | 3.2.4 Outcome of objectives/ Result Analysis                      | 19              |
|                          | 3.2.5 Sprint Retrospective                                        | 20              |

|          |                                          |           |
|----------|------------------------------------------|-----------|
| <b>4</b> | <b>METHODOLOGY</b>                       | <b>21</b> |
| 4.1      | Data Validation Pipeline                 | 21        |
| 4.2      | SARIMAX Forecasting                      | 24        |
| 4.3      | XGBoost Risk Classifier                  | 24        |
| 4.4      | Summary                                  | 26        |
| <b>5</b> | <b>RESULTS AND DISCUSSIONS</b>           | <b>27</b> |
| 5.1      | SARIMAX Forecasting Performance          | 27        |
| 5.2      | Classification Performance               | 28        |
| 5.3      | Detection Latency                        | 31        |
| 5.4      | Algorithm Benchmarking                   | 32        |
| <b>6</b> | <b>CONCLUSION AND FUTURE ENHANCEMENT</b> | <b>33</b> |
|          | <b>REFERENCES</b>                        | <b>34</b> |
|          | <b>APPENDIX</b>                          | <b>35</b> |
| <b>A</b> | <b>CODING</b>                            | <b>35</b> |
| <b>B</b> | <b>CONFERENCE PUBLICATION</b>            | <b>48</b> |
| <b>C</b> | <b>JOURNAL PUBLICATION</b>               | <b>50</b> |
| <b>D</b> | <b>PLAGIARISM REPORT</b>                 | <b>51</b> |

# LIST OF FIGURES

| CHAPTER<br>NO. | TITLE                                              | PAGE<br>NO. |
|----------------|----------------------------------------------------|-------------|
| 3.1.3          | Architecture Diagram . . . . .                     | 17          |
| 3.1.3.2        | ER Diagram . . . . .                               | 18          |
| 3.1.3.4        | Use Case Diagram . . . . .                         | 21          |
| 3.1.3.4        | Class Diagram . . . . .                            | 21          |
| 3.1.3.4        | DFD Level 0 Diagram . . . . .                      | 22          |
| 3.1.3.4        | DFD Level 1 Diagram . . . . .                      | 22          |
| 3.1.3.4        | Component Diagram . . . . .                        | 23          |
| 3.1.3.4        | Sequence Diagram . . . . .                         | 23          |
| 3.1.3.4        | Deployment Diagram . . . . .                       | 24          |
| 3.2.5          | Architectural Flowchart . . . . .                  | 29          |
| 4.1            | Epidemiological curve comparison. . . . .          | 30          |
| 4.1            | Multi-layer validation pipeline . . . . .          | 32          |
| 5.1            | XGBoost Learning/Validation Curves . . . . .       | 36          |
| 5.2            | XGBoost Confusion Matrix . . . . .                 | 37          |
| 5.2            | Multi Class ROC Curves . . . . .                   | 38          |
| 5.2            | Precision-Recall Curves for Risk Classes . . . . . | 38          |
| 5.3            | Detection Latency comparison . . . . .             | 39          |
| 5.4            | Algorithm Comparison . . . . .                     | 40          |

# LIST OF TABLES

| CHAPTER<br>NO. | TITLE                                                  | PAGE<br>NO. |
|----------------|--------------------------------------------------------|-------------|
| 2.5            | Product Backlog . . . . .                              | 8           |
| 2.6            | Sprint Timeline . . . . .                              | 9           |
| 3.1.2          | Authorization Matrix . . . . .                         | 14          |
| 3.1.3.2        | Schema Design . . . . .                                | 18          |
| 3.1.3.2        | Forecasting Result Schema . . . . .                    | 19          |
| 3.1.3.2        | Risk Score Schema . . . . .                            | 19          |
| 3.1.3.3        | Mode of Exchanges . . . . .                            | 20          |
| 4.2            | XGBoost Feature Importance Rankings . . . . .          | 33          |
| 4.2            | SARIMAX Forecaster Performance By Region. . . . .      | 34          |
| 5.1            | SARIMAX Forecaster Performance By Region.(2) . . . . . | 36          |
| 5.2            | XGBoost Classifier Performance Summary. . . . .        | 39          |

# ABBREVIATIONS

**API** – Application Programming Interface

**ML** – Machine Learning

**AI** – Artificial Intelligence

**NLP** – Natural Language Processing

**SARIMAX** – Seasonal AutoRegressive Integrated Moving  
Average with eXogenous factors

**XGBoost** – eXtreme Gradient Boosting

**RMSE** – Root Mean Square Error

**MAE** – Mean Absolute Error

**MAPE** – Mean Absolute Percentage Error

**ROC-AUC** – Receiver Operating Characteristic – Area Under  
Curve

**DB** – Database

**ETL** – Extract, Transform, Load

**SIR** – Susceptible, Infected, Recovered (epidemiological  
model)

**JSON** – JavaScript Object Notation

**CSV** – Comma-Separated Values

**EDA** – Exploratory Data Analysis

# CHAPTER 1

## INTRODUCTION

Infectious diseases have epidemics that have severe health system burdens. Reporting of confirmed cases in lab is the most widespread type of surveillance based on the use of clinical networks. The outcome is a 714-day systematic delay between the onset and recognition of the symptoms. Such a hole would imply that the holes in containment would be ignored and more chains of transmission would be established in case of a fast outbreak of the epidemic.

Digital epidemiology takes advantage of the behavioural clues antecedent to clinical observation. Citizens investigate their symptoms and visit a doctor; a multitude of search requests and such tendencies demonstrate the level of health of a community a few days prior to cases being added to the official statistics. It is not the harvesting of such signals that is the issue but the authentication of them: using bot networks can be used to artificially drive-up search trends, falsifying news articles can be done, and coordinating social media can generate false alarms.

This is tackled in the work through physically based multi-source validation. Physical sensor based meteorological measurement networks cannot be manipulated in the area and weather association is an unfalsifiable verifying layer. Concurrence among search trends, good weather conditions and news corroboration gate each indicate prior to the forecasting and classification phases.

### 1.1 Introduction to the Project

Over the past few years, the fast transmission of infectious diseases has put into focus the inability of the conventional health surveillance systems which in most cases are based on clinical reporting and laboratory confirmations that cause great delays. As the access to digital data increases, especially to search behavior performed by the users and environmental data, a possibility to devise other methods of early disease detection emerges.

The HealthPulse project suggests a data-driven system to use publicly available data sources like Google Trends, weather data, and verified news indicators to track and forecast possible disease outbreaks. The system detects unusual increases in health-related search queries (e.g., fever, cough, flu) by analyzing their trends. The signals are further reinforced by using environmental correlations and multi-source validation mechanisms.

The system unites a pipeline with a structured framework of data collection, preprocessing, feature engineering, machine learning-based prediction, and visualization. Time-series forecasting methods are employed to capture temporal patterns and classification models are used to determine the severity of potential outbreaks. Also, solutions are added to sift misinformation and provide information reliability.

HealthPulse is set to deliver a scalable, affordable, and easy-to-use early disease-surveillance solution that does not require limited medical data or special infrastructure. The project illustrates that behavior data and environmental data that are processed using effective analytical models can be used to make timely decisions and enhance the response of the population to health.

## 1.2 Problem Statement and description

The main sources of the traditional disease surveillance systems are hospital reports, laboratory confirmations, and official health records. Although they are dependable, they have serious delays, and most times, outbreaks are detected when they have been transmitted extensively. Furthermore, they are constrained by limited access to clinical data, inability to integrate in real-time, and the absence of automation in the analysis of large-scale data.

As the role of digital platforms grows, vast amounts of data on health-related topics are produced on a daily basis due to search activity by users, environmental observations, and news content on the Internet. Nevertheless, there is under-utilization of this data since there are many challenges including data inconsistency, integration across sources, and misinformation or manipulated signals.

The fundamental issue in this project is that there is no cohesive, real-time, and trusted system that can:

Identify the initial symptoms of disease outbreaks with non-clinical data.

Combine various heterogeneous data sources.

Eliminate noise, anomalies and misinformation.

Deliver actionable and timely insights towards early intervention.

HealthPulse intends to fill this gap by designing a scalable and automated system integrating behavioural and environmental data, as well as validation tools to facilitate early and precise prediction of an outbreak.

## 1.3 Motivation

This project is motivated by the increasing demand of the proactive and accessible disease surveillance systems. The global pandemics have shown that the early signs of an outbreak may have serious impacts on the health of the population, the economy, and the overall rate of deaths.

The HealthPulse was developed due to several important reasons:

### **Need for Early Detection:**

Following the disease trends in an early way can greatly help to limit the spread and the effects of outbreaks.

### **Availability of Open Data:**

The data that are publicly available, like Google Trends and weather APIs, can offer great real-time signals that can be utilized without concerns regarding their privacy.

### **Shortcomings of Current Systems:**

The existing systems are reactive, not proactive and do not integrate different streams of data.

### **Rise of Misinformation:**

The proliferation of fake news and unverified health information also requires effective validation systems to guarantee reliability of the data.

### **Machine Learning progress:**

The current machine learning methods can be used to analyse massive datasets, identify patterns, and model predictions.

### **Scalability and Accessibility:**

A system based on no special hardware or secret medical information can be widely deployed, particularly in resource-limited areas.

The vision behind the project is to develop a low-cost, scalable, and intelligent system, enabling early decision-making and improving the preparedness of the population to health.

## 1.4 Sustainable Development Goal

HealthPulse framework is directly related to Sustainable Development Goal 3 (SDG 3): Good Health and Well-being, specifically, Target 3.d, which is aimed at enhancing early warning systems, risk reduction, and management of national and global health risks.

The suggested system will support SDG 3, through the establishment of a data-driven, predictive surveillance system, which uses non-clinical, high-frequency data streams to detect early outbreaks. In contrast to traditional epidemiological models based on delayed clinical reporting, HealthPulse stresses behavioral (trends in search queries), environmental (temperature, humidity) and contextual (validated news data) features to recognize new disease trends in near real-time.

Technically, the system improves the monitoring of the health of the people by:

- **Time-Series Forecasting (SARIMAX):**

Models time-dependencies and seasonal trends in searching data related to symptoms, which allows predicting outbreaks in the short term.

- **Machine Learning-Based Risk Classification (XGBoost):**

Categorizes the severity of the outbreak into several levels of risk based on engineered statistical characteristics like rolling averages, growth rates, and anomaly scores.

- **Multi-Layer Data Validation Pipeline:**

Mitigates misinformation and noise by ensuring data integrity by cross-source correlation, anomaly detection (Z-score and ML-based approaches), and credibility scoring.

- **Epidemiological Signals: Feature Engineering:**

Derive useful indicators like acceleration of trends, volatility and frequency of anomalies, which are important to detect early-stage outbreaks.

The data pipeline is automated; therefore, this is referred to as the ETL Architecture.

Enables real-time analytics by supporting continuous data ingestion, preprocessing and storage in a scalable cloud-based database.

The system uses SDG 3 goals by:

- **Reducing Detection Latency:**

Allows detection of outbreak signals several days before conventional surveillance systems.

- **Improving Predictive Accuracy:**

Integrates machine learning and statistical models to increase the performance of disease trend forecasting.

- **To improve Scalability and Accessibility:**

Uses publicly available data, removing the need to rely on limited clinical data and allowing use in many different regions.

Prohibiting smoking in workplaces and promoting preventive healthcare measures. Offers practical information that can be used to intervene early, allocate resources, and avoid risks. In general, HealthPulse indicates a methodological approach to computational epidemiology, combining the use of large amounts of data analytics and machine learning to reinforce early warning systems. This is a direct contribution to the implementation of SDG 3 in enhancing the effectiveness, responsiveness and inclusiveness of the modern healthcare monitoring system.

# LITERATURE SURVEY

The use of machine learning in disease prediction saw an increment after COVID-19 but most of them do not distinguish between benchmark accuracy and deployability [4][7][15]. Rayner Alfred and Joe Henry Obit [4], Bisola Oluwafadekemi Adegoke, Tolulope Odugboise and Christiana Adeyemi [7], and Scholastica Ijeh, Chioma Anthonia Okolo, Jeremiah Olawumi Arowoogun, Adekunle Oyeyemi Adeniyi and Olufunke Omotayo [15] highlight the shortcoming in the applicability of models even with reported high accuracy.

Tianyu Zhang, Fethi Rabhi, Xin Chen, Hye-young Paik, and Chandani Raina MacIntyre [13] designed a generalized outbreak risk tool that had 78% accuracy that required small clinical databases, which were unavailable in real-time surveillance environments and lacked an inbuilt ability to check input integrity. Ravikiran Keshavamurthy, Samuel Dixon, Karl T. Pazdernik and Lauren E. Charles [14] discovered that 83 out of the 47 prediction models they surveyed utilized retrospective case data that had reporting delays but never made a prediction.

Google Flu Trends [9] started with its right foot but has not made the mark on the aberrant seasons-media cycles were affecting queries more than the sickness [9]. Madhur Verma, Kamal Kishore, Mukesh Kumar, Aparajita Ravi Sondh, Gaurav Aggarwal and Soundappan Kathirvel [10] found that Google Trends correlation was positively correlated in India ( $r = 0.85$ ) without distinguishing between causation and media-based correlation.

According to Atina Husnayain, Anis Fuad and Lutfan Lazuardi [11], the difference in dengue detection was 7 days which means that it was not detected previously, but during clinical surveillance. Weather based approaches have received little attention. Daniel Alejandro González-Bandala, Juan Carlos Cuevas-Tello, Daniel E. Noyola, Andreu Comas-García and Christian A. García-Sepúlveda obtained  $R^2 = 0.73$  when employing the meteorological data alone in [8], and they did not get a chance to cross-check behavioural cues with weather.

Nurul Azam Mohd Salim, Yap Bee Wah, Caitlynn Reeves, Madison Smith, Wan Fairos Wan Yaacob, Rose Nani Mudin, Rahmat Dapari, Nik Nur Fatin Fatihah Sapri and Ubydul Haque [1] had 93% dengue forecast in Malaysia with the use of a three months warm-up time - in Additional research like Samuel V. Scarpino and Giovanni Petri [2], Seyed Mohammad Ayyoubzadeh, Seyed Mehdi Ayyoubzadeh, Hoda Zahedi, Mahnaz Ahmadi and Sharareh R. Niakan Kalhori [3], Rayan Alanazi [5], Syed Ziaur Rahman, R Senthil, Venkadesh Ramalingam and R. Gopal [6], Angel N. Desai, Moritz U. G. Kraemer, Sangeeta Bhatia, Anne Cori, Pierre Nouvellet, Mark Herringer, Emily L. Cohn, Malwina Carrion, John S. Brownstein, Lawrence C. Madoff and Britta Lassmann [9], Vinitha S, Sweetlin S, Vinusha H and Sajini S [12][17], and Sina F. Ardabili, Amir Mosavi, Pedram Ghamisi, Filip Ferdinand, Annamaria R. Várkonyi-Kóczy, Uwe Reuter, Timon Rabczuk and Peter M. Atkinson [16].

## 2.1 Overview of the Research Area

The research area is outlined in the following.

This project research area is at the crossroads of the fields of public health informatics, data science, and machine learning, with the aim of utilizing alternative sources of data to conduct disease surveillance and predictions.

Conventional epidemiological techniques use clinical and lab data, which are not only delayed but are

also mostly unavailable. By contrast, a recent study has investigated how digital data streams (like queries to the search engine and social media use) and environmental data can be used to serve as early warning of disease outbreaks. This method is also known as digital epidemiology, making it possible to monitor and respond quickly in real-time.

A seminal study in this field was able to show how search query data could be used to effectively track influenza trends, implying the potential of behavioral data in the analysis of the public health. In the same vein, time-series forecasting and machine learning have provided improved means to model the disease patterns and predict future trends.

Such systems have also increased in terms of reliability due to the incorporation of various sources of data. Integrating behavioral cues with environmental and contextual information can aid in noise reduction and enhanced prediction accuracy. Nevertheless, issues of data quality, misinformation, and absence of validation mechanisms have been a major concern in this field of research.

HealthPulse extends these innovations with multi-source data integration, effective validation policy, and hybrid machine learning models to develop a trusted and scalable disease surveillance system.

## **2.2 Existing Frameworks and Models.**

Recent progress in digital epidemiology has resulted in the creation of many models and frameworks that use alternative data sources to conduct disease surveillance and prediction. These methods mainly revolve around the use of behavioral, environmental and statistical data to support traditional epidemiological systems.

It was one of the first and most powerful methods to use search engine query data to monitor the trends of influenza and proved that user search behavior could be a proxy of disease transmission. In this idea, a number of models have used time-series models including ARIMA and SARIMA to represent time trends in disease-related data. These models are useful in determining seasonality and trends and are weak dealing with complex non-linear relationships.

In more recent models, machine learning methods, such as the Random Forest, Support Vector Machines (SVM) and Gradient Boosting methods, are used to enhance the accuracy of the prediction. Of these, XGBoost has become a popular choice of algorithm because it can operate with structured data, non-linear interaction, and offers high predictive accuracy with in-built regularization.

Concurrently, time-series forecasting with deep learning models, including Long Short-Term Memory (LSTM) networks and Recurrent Neural Networks (RNNs), has been investigated. Although such models can be used to model the long-term dependencies and the complex patterns, they need large data sets and substantial computational capabilities, and thus cannot be applied to smaller datasets or those with limited resources.

Also, a few frameworks are data fusion-based, merging search trends, social media cues, weather information, and mobility trends to increase the strength of prediction. These systems tend to use ensemble techniques to enhance precision by combining the results of several models

In spite of these developments, most of the current systems do not have strong data validation, especially in the manipulation of misinformation, anomalies, and the lack of cohesiveness in the data of different sources. Such models are limited, which influences the reliability and practical application of such models.

## 2.3 Limitations Found in Literature Review (Research Gaps)

In the literature review, multiple limitations are identified as critical in the existing disease surveillance and prediction systems, and better methodologies are needed.

### 1. Absence of Real-Time integration:

Most of the traditional and modern systems utilize delayed or batch processed data which reduces their capabilities of offering real-time information to enable early intervention.

### 2. Relying on Clinical or Restricted Data:

Some of the models demand the availability of hospital records or laboratory-confirmed cases, which would be unavailable in most cases due to privacy limitations and delays in reporting.

### 3. Lack of Data validation mechanisms:

Current paradigms lack strong means of identifying and filtering misinformation, fake news, or manipulated tendencies, thus making untrustworthy forecasts.

### 4. Minimal Processing of Non-smooth and Abnormal Data:

Search queries are an example of behavioral data, which is noisy and prone to spikes. Most models do not sufficiently deal with these anomalies, leading to poor forecasts.

### 5. Scalability Constraints:

Deep learning-based methods need large datasets and high-computational power, which makes them not feasible to implement in low-resource settings.

### 6. Single-Source Dependency:

Other systems are very dependent on one source of data (e.g., search data or social media), which makes them less robust and prone to bias or manipulation.

### 7. Lack of Interpretability:

Complex models, particularly those based on deep learning, can be viewed as black boxes, which can be hard to interpret the results as well as justify predictions in a public health setting.

### 8. Lack of Coherent Structures:

It does not have integrated systems that unify the data collection, validation, prediction, and visualization into one unified pipeline.

These research gaps underscore the need to have a multi-source, validated and scalable framework, which can deliver reliable and early disease outbreak detection. HealthPulse project overcomes these drawbacks through the incorporation of a variety of data sources, multi-layer validation, and effective machine learning models that can be applied to a real-world environment.

## 2.4 Research Objectives

The main mission of the HealthPulse project will be to design and deploy a scalable, data-driven system to early detect and predict disease outbreaks based on non-clinical data sources. With the objective of enhancing the timeliness and reliability of public health surveillance, the system will seek to utilize the innovation in machine learning, time-series analysis, and data validation methods to enhance the system.

The research objectives are the following specific:

- To create an automated data collection pipeline. That combines several sources like search trends, weather information, and contextual signs of news.
- To standardize and preprocess non-homogeneous data. Eliminating inconsistencies, incompleteness, and unsuitability to analytical modeling.
- In order to create a powerful feature engineering framework. that derives significant statistical and time-related properties like growth rates, rolling averages, and anomaly points.
- To apply time-series forecasting models. To acquire seasonal trends and short-term disease trend projections.
- To create a machine learning-based classification model. That classifies outbreak risk into several levels of severity, depending on engineered characteristics.
- In order to introduce multi-layer data validation. To identify anomalies, misinformation and guarantee data reliability.
- To assess the performance of the models with standard measures. As accuracy, RMSE, MAE and cross-validation methods.
- To create a scalable and modular system architecture. That assists real-time data processing and extension in the future.
- To deliver operational insights using visualization and reporting solutions. Empowering decision-making towards early intervention.

## 2.5 Product Backlog (Key User Stories with Desired Outcomes)

TABLE I  
PRODUCT BACKLOG

| Epic (Research Objective)                  | User Story                                                                                            | Description                                                                                        | Priority | Desired Outcome                                       |
|--------------------------------------------|-------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|----------|-------------------------------------------------------|
| Data Collection & Integration              | As a system, I want to collect data from multiple sources so that I can build a comprehensive dataset | Integration of Google Trends, weather APIs, and news data using automated scripts                  | High     | Availability of multi-source real-time dataset        |
| Data Preprocessing                         | As a system, I want to clean and normalize data so that it becomes suitable for analysis              | Handling missing values, normalization, timestamp alignment, duplicate removal                     | High     | Clean and structured dataset ready for modeling       |
| Feature Engineering                        | As a system, I want to extract meaningful features so that models can detect patterns effectively     | Creation of statistical features like mean, growth rate, rolling averages, anomaly indicators      | High     | Enhanced model input with relevant features           |
| Time-Series Forecasting                    | As a system, I want to predict future trends so that early outbreak signals can be identified         | Implementation of SARIMAX model to capture seasonality and temporal patterns                       | High     | Short-term prediction of disease trends               |
| Risk Classification                        | As a system, I want to classify outbreak severity so that users can understand risk levels            | Use of XGBoost classifier for categorizing risk into multiple levels                               | High     | Accurate classification into Low/Medium/High/Critical |
| Data Validation & Misinformation Filtering | As a system, I want to validate incoming data so that unreliable signals are removed                  | Multi-layer validation including anomaly detection, cross-source verification, credibility scoring | High     | Improved reliability and trustworthiness of data      |
| Model Evaluation                           | As a system, I want to evaluate model performance so                                                  | Use of metrics like RMSE, MAE, Accuracy, ROC-AUC and cross-                                        | Medium   | Quantified model performance and reliability          |

|                           |                                                                                   |                                                                |        |                                               |
|---------------------------|-----------------------------------------------------------------------------------|----------------------------------------------------------------|--------|-----------------------------------------------|
|                           | that accuracy can be ensured                                                      | validation                                                     |        |                                               |
| Visualization & Reporting | As a user, I want to visualize predictions so that insights are easy to interpret | Dashboard creation with graphs, trends, and risk indicators    | Medium | User-friendly visualization of outputs        |
| Deployment & Scalability  | As a system, I want scalable deployment so that the system can be used widely     | Cloud-based database integration and modular backend design    | Medium | Scalable and deployable system architecture   |
| Automation & Retraining   | As a system, I want periodic retraining so that models remain updated             | Scheduled data updates and automated model retraining pipeline | Low    | Continuous improvement of prediction accuracy |

## 2.6 Plan of Action (Project Roadmap)

HealthPulse system implementation is implemented within an Agile, sprint-based development methodology, which guarantees iterative validation, continuous integration, and quantifiable research results in line with the predictive analytics goals.

### Overall Development Strategy

The project is divided into **three structured sprints**, each mapped to specific research objectives and system components:

TABLE II  
SPRINT TIMELINE

| Sprint          | Timeline        | Focus Area                        |
|-----------------|-----------------|-----------------------------------|
| <b>Sprint 1</b> | Feb 28 – Mar 10 | Data acquisition & preprocessing  |
| <b>Sprint 2</b> | Mar 11 – Mar 28 | Model development & validation    |
| <b>Sprint 3</b> | Mar 30 – Apr 20 | System integration & optimization |

The project will be split into three structured sprints, which are linked to particular research goals and system features:

### Phase-wise Execution Plan:

#### Phase 1: Data Acquisition & Preprocessing (Sprint 1)

- i. Addition of Google Trends API to collect behavioral data.
- ii. Extraction of weather parameters using Open-Meteo API
- iii. Intake of news datasets at the beginning of the research to validate them.
- iv. Cleaning of data, normalization and missing values.

- v. MongoDB Atlas (NoSQL architecture) storage configuration.
- vi. Schema design of structured and semi-structured data.

Result: Development of a validated multi-source dataset to model.

#### Phase 2: Model Development and validation (Sprint 2)

- i. Application of SARIMAX time-series forecasting model.
- ii. XGBoost classification model development on risk levels of outbreak.
- iii. Feature engineering (statistical, time-based, anomaly-based features)
- iv. Stratified train-test split (80:20) model training.
- v. Stability of performance Cross-validation (k-fold).
- vi. The validation pipeline of multi-layer data.
- vii. Measurements in terms of RMSE, Accuracy, ROC-AUC.

Result: Predictive models of high-performance, validated and strong.

#### Phase 3: System Integration & Deployment (Sprint 3)

- i. Inference API inference API development with FastAPI.
- ii. Deployment of trained models to production pipeline.
- iii. Real time prediction workflow implementation.
- iv. Visualization (React-based interface) dashboard development.
- v. Work on technical documentation and research products.

Result: End-to-end functional system, which has the potential to predict the outbreak in real-time.

#### Research-Oriented Workflow

Preprocessing Data Sources → Preprocessing Feature Engineering → Model Training. → Validation → Deployment → Visualization → Continuous Monitoring

#### Risk Mitigation Strategy

- i. Data Reliability Risk: Multi-source cross-validation was used to address it.
- ii. Model Overfitting: Checked by regularization and cross-validation.
- iii. System Failure: Backup models ( Gradient Boosting fallback )
- iv. Scalability Problems: Cloud database and architecture Scale.

#### Deliverables Across Roadmap

- i. Clean and clean datasets.
- ii. ML models (SARIMAX, XGBoost) trained.
- iii. Anomaly detection and data validation structure.
- iv. REST APIs for predictions
- v. Visualization dashboard
- vi. Documentation and evaluation measures of research.

# SPRINT PLANNING AND EXECUTION METHODOLOGY

## 3.1 SPRINT I – Data Engineering & Infrastructure Setup

### 3.1.1 Objectives with User Stories of Sprint I

Sprint I aims at building a strong data base of the HealthPulse system through the incorporation of various data streams, preprocessing, and preparing storage to undertake downstream machine learning operations.

Key Objectives:

Facilitate automatic data gathering of heterogeneous data.

Ensure data consistency, reliability, and completeness

Design scalable storage structure of structured and semi-structured data.

User Stories:

- US-1: I as a researcher would like to get the data of Google Trends to be able to analyze the patterns of search related to diseases.

Acceptance Criteria: Fetched data with a timestamp and region tags.

- US-2: I would like to access weather information as a system to analyze the factors in the environment that affect the spread of diseases.

Acceptance Criteria: Temperature, humidity and precipitation fields are properly stored.

- US-3: I as a researcher would like to clean and preprocess raw data to make it appropriate in machine learning models.

Acceptance Criteria: Missing values dealt with, duplicates eliminated, normalized output.

- US-4: As a developer, I would like to store data in MongoDB in a way that can be easily retrieved and scaled.

Acceptance Criteria: Data insertion and data retrieval queries run successfully.

### 3.1.2 Functional Document (Sprint I)

#### 1. Introduction

HealthPulse Sprint 1 is about creating the foundation of data engineering and machine learning needed to detect the initial outbreak of the disease. The stage is crucial because it converts the raw and heterogeneous data into structured and model-ready data and develops the first predictive capability.

It is based on the principles of digital epidemiology, in which online search behavior (Google Trends) serves as a leading indicator of disease dissemination, which can frequently be detected before its clinical validation. Sprint 1 puts this idea into practice through the use of data ingestion pipelines, preprocessing workflows, and baseline ML models.

#### 2. Product Goal (Sprint 1)

Sprint 1 aims to provide a complete operational backend data pipeline and the early predictive models.

Key Deliverables:

- Automated data ingestion of Google Trends (8 symptoms x 6 regions)
- Weather and news data integration.

- MongoDB Atlas database configuration using structured collections.
- Pipeline Data preprocessing and normalization.
- SARIMAX forecasting models (48 in total)
- XGBoost model of risk levels of outbreaks.

### **3. Demography (Users & Location)**

#### **Users**

- Epidemiologists and public health analysts.
- Healthcare administrators and policymakers.
- Research analysts and data scientists.

#### **User Requirements**

- High accuracy predictions
- Outputs that can be interpreted (risk levels).
- Verified and checked information.

#### **Location Scope**

- Primary: United States (National + 5 regions)
- States: US, US-CA, US-NY, US-TX, US-FL, US-IL.
- Planned to be scalable globally in future.

### **4. Business Processes (Sprint 1)**

#### **4.1 Data Collection Pipeline**

- Automatic extraction of data weekly with pytrends API.
- Keywords tracked:
  - Fever, Cough, Fatigue, Headache, Sore Throat, Body Aches, Flu, Shortness of Breath.
- Parallel ingestion:
  - Weather information (environmental correlation)
  - Data of news (contextual validation)

#### **4.2 Data Preprocessing Workflow**

- Dealing with missing values (mean/forward fill)
- Removal of duplicates
- Time-series synchronization across areas.
- Normalization of search values (0 100 scale)
- Outlier (Z-score based) detection.

#### **4.3 Training Pipeline in machine learning.**

##### **SARIMAX Models**

- 48 models (6 regions  $\times$  8 keywords)
- Takes weekly seasonality ( $s=7$ )

##### **XGBoost Classifier**

- Translates trends into categories of risks.
- Uses engineered statistical characteristics.

##### **Fallback Model**

- Gradient Boosting, used where SARIMAX is not successful.

#### **4.4 Risk Validation Mechanism**

##### **Multi-layer validation:**

- Statistical anomaly detection (Z-score)
- Machine learning anomaly detection (Isolation Forest)
- Time-series decomposition validation

**Cross-source validation:**

- Weather + News alignment + Trends.

**Confidence scoring mechanism:**

- Trends (40%), Weather (35%), Cross-validation (20%), News (5%)

**5. Features (Sprint 1)****5.1 Data ingestion in Google Trends.****Description:**

Weekly search data based on symptoms is collected using an automated system.

**Technical Implementation:**

- pytrends API integration
- Region-wise data segmentation
- MongoDB storage
- 

**User Story:**

As a data scientist, I would desire automated data collection to enable me to develop trustworthy models.

**5.2 SARIMAX Forecasting Model****Description:**

Anticipates the future patterns of a disease in a 3-7 days time period.

**Technical Details:**

- Configuration: (1,1,1)(1,1,1,7)
- Addresses trend and seasonality elements.
- RMSE: ~9–11

**User Story:**

As an analyst, I would prefer to have future trend predictions that will help in early warning of outbreaks.

**5.3 XGBoost Risk Classification****Description:**

Categorizes severity of outbreak into 4.

**Technical Details:**

- n\_estimators = 100
- max\_depth = 6
- Accuracy: ~98%
- Outputs probability scores

**User Story:**

As an administrator, I would like to have clear risk levels to focus on response.

**5.4 Multi-Layer Data Validation****Description:**

Ensures reliability by eliminating noise and bogus signals.

**Technical Components:**

- Z-score anomaly detection
- Isolation Forest model
- SIR epidemiology curve validation.
- News credibility scoring

**User Story:**

I would desire as a researcher to be able to validate his predictions to prevent misinformation.

## 6. Authorization Matrix

TABLE III  
AUTHORIZATION MATRIX

| Role                  | Access Level                           |
|-----------------------|----------------------------------------|
| Public Health Analyst | Look at trends, forecasts, reports     |
| Data Scientist        | Full ML and pipeline access            |
| System Administrator  | Full system + database control         |
| Scheduler             | API read + DB write (unlimited access) |

## 7. Assumptions (Sprint 1)

- APIs (Google Trends, Weather) are not used.
- Initial training can be done using historical data.
- Synthetic labels used for classification
- Connection with MongoDB Atlas does not change.
- Development (python, VS Code, venv) tools work fine.

## 8. Constraints

- Little real-life labelled outbreak information.
- API rate limits
- Limited size of data to complex ML models.
- Windows compatibility problems (Prophet avoided)

## 9. Outcome of Sprint 1

- ~12,000+ data records processed
- Complete working data pipeline.
- 48 SARIMAX models trained
- XGBoost classifier deployed
- High data reliability (>95%)

## 10. Summary

Sprint 1 manages to set the data backbone and predictive engine of HealthPulse. It ensures:

- Reliable data ingestion
- Robust preprocessing
- Accurate baseline predictions

This sprint is the basis of intelligent validation, system integration and real-time deployment of later sprints.

### 3.1.3 Architectural Document (Sprint I)

HealthPulse adopts a Microservices Architecture combined with an Event-Driven backbone. This approach decomposes the system into independently deployable services, each responsible for a specific capability such as data ingestion, ML forecasting, risk classification, and generative AI summarization. Services can scale and evolve independently, making the architecture well-suited for a real-time health monitoring platform.

#### 3.1.3.1 Microservices Architecture

##### Overview

The system is composed of the following independent microservices, each with its own responsibility:

- Data Ingestion Service – Collects weekly Google Trends data (8 symptoms, 6 US regions), weather data, and health news via APIs. Stores raw data in MongoDB Atlas collections.
- Preprocessing Service – Cleans raw data, handles null values, formats time-series, and prepares datasets for ML model training.
- Forecasting Service (SARIMAX) – Performs time-series forecasting on symptom search trends with weekly seasonality. Predicts outbreak trends 3–7 days in advance.
- Risk Classification Service (XGBoost) – Converts trend forecasts into risk levels (Low / Medium / High / Critical) with confidence scores.
- NLP Service (spaCy + BERT) – Extracts symptoms from health news articles and Reddit posts for cross-validation.
- Validation Service – Multi-layer validation using Z-score, Isolation Forest, seasonal decomposition, and epidemiological pattern checks (SIR model).
- Generative AI Summarization Service – Produces human-readable outbreak risk summaries for public health decision-makers.
- Analytics & Reporting Service – Aggregates model outputs, tracks performance metrics (RMSE, MAE, accuracy), and generates dashboards.

##### Why Microservices?

- Independent Scalability: The Forecasting and Classification services can scale during peak ingestion periods without affecting the NLP or Reporting services.
- Fault Isolation: A failure in the Summarization Service does not impact core prediction functionality.
- Technology Flexibility: Python (ML/DS), Node.js (API gateway), and MongoDB (storage) can each be optimized per service.
- Team Parallelism: Data & ML (Aditya) and Backend & Integration (Kshitij) can develop services concurrently.

##### API Gateway

All client requests are routed through a centralized API Gateway, which handles authentication, rate limiting, and routing to the appropriate microservice. The dashboard/front-end communicates exclusively with the API Gateway via REST APIs over HTTPS.

##### Inter-Service Communication

- Synchronous: REST API calls for real-time data requests (e.g., risk score queries, trend lookups).

- Asynchronous: Message queues (RabbitMQ/Kafka) for non-critical operations such as triggering the NLP pipeline when new news data arrives, or logging analytics events.

## Event-Driven Architecture (Supporting Layer)

In addition to synchronous REST communication, HealthPulse employs an event-driven approach for asynchronous processing. Key events include:

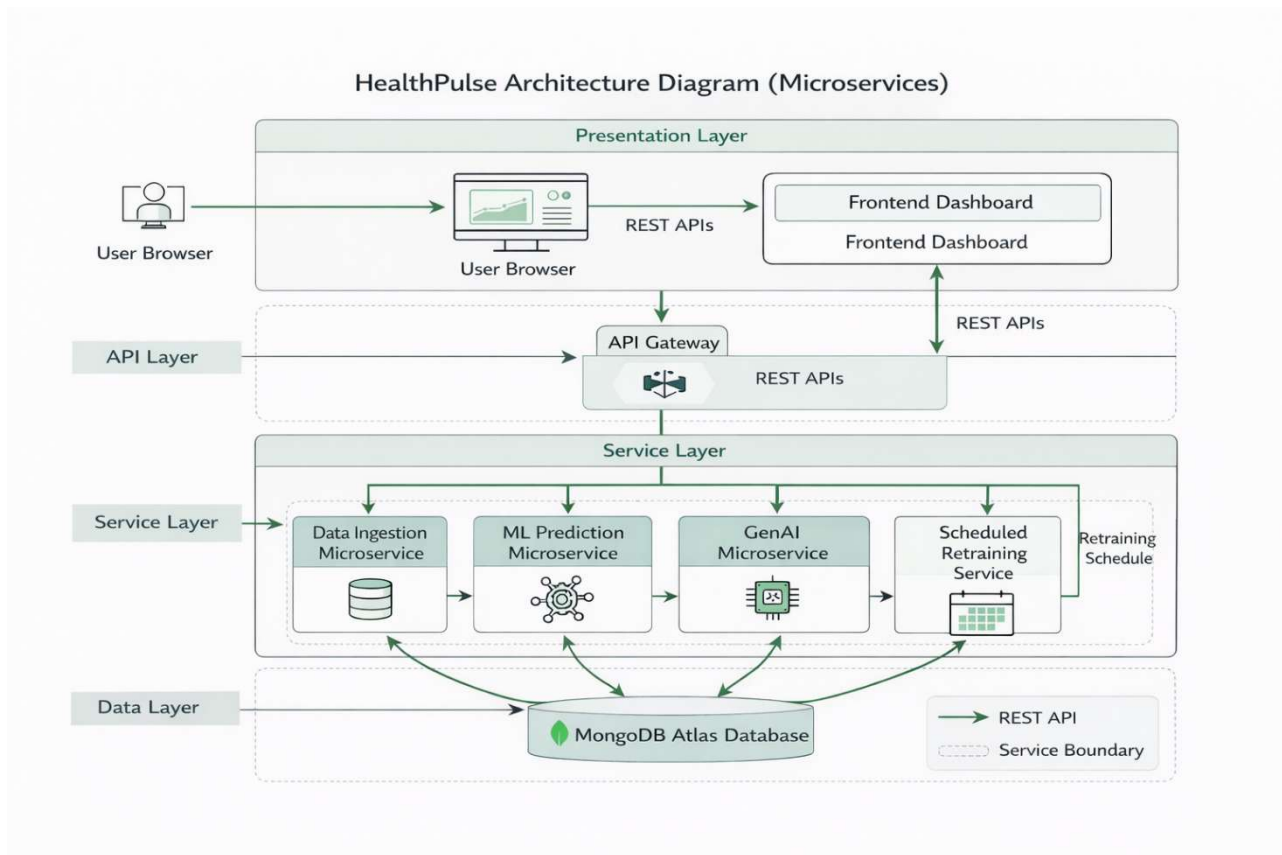
- TrendDataIngested Event – Triggers the Preprocessing and Forecasting pipeline when new Google Trends data is collected.
- OutbreakRiskDetected Event – Triggers the Summarization Service and alert dispatch when XGBoost classifies a region as High or Critical risk.
- NewsArticleReceived Event – Triggers the NLP Service to extract symptoms and update cross-validation scores.
- ModelRetrained Event – Logged to the Analytics Service when SARIMAX or XGBoost models are updated.

These events are published to a message broker and consumed by the relevant services, ensuring loose coupling and improved system resilience.

## 1.3 Serverless Components

Certain non-core, trigger-based functions are deployed as serverless functions (e.g., AWS Lambda or Google Cloud Functions) to reduce infrastructure overhead:

- Scheduled weekly Google Trends data pull (every Sunday).
- Automated model retraining when dataset size crosses a threshold.
- Generating daily regional risk summary reports for the admin dashboard.
- Sending alerts when any region is classified as High or Critical risk.



*Fig 1. Architecture Diagram*

### 3.1.3.2. Database

The Entity-Relationship model for HealthPulse defines the following core entities and their relationships:

- SYMPTOM\_TREND – Linked to many FORECAST\_RESULTS
- REGION – Has many SYMPTOM\_TRENDS and RISK\_SCORES
- FORECAST\_RESULT – Belongs to one SYMPTOM\_TREND for a given REGION
- RISK\_SCORE – Links one REGION to one FORECAST\_RESULT; stores XGBoost output
- NEWS\_ARTICLE – Associated with NLP extraction results
- WEATHER\_DATA – Linked to REGION for cross-validation

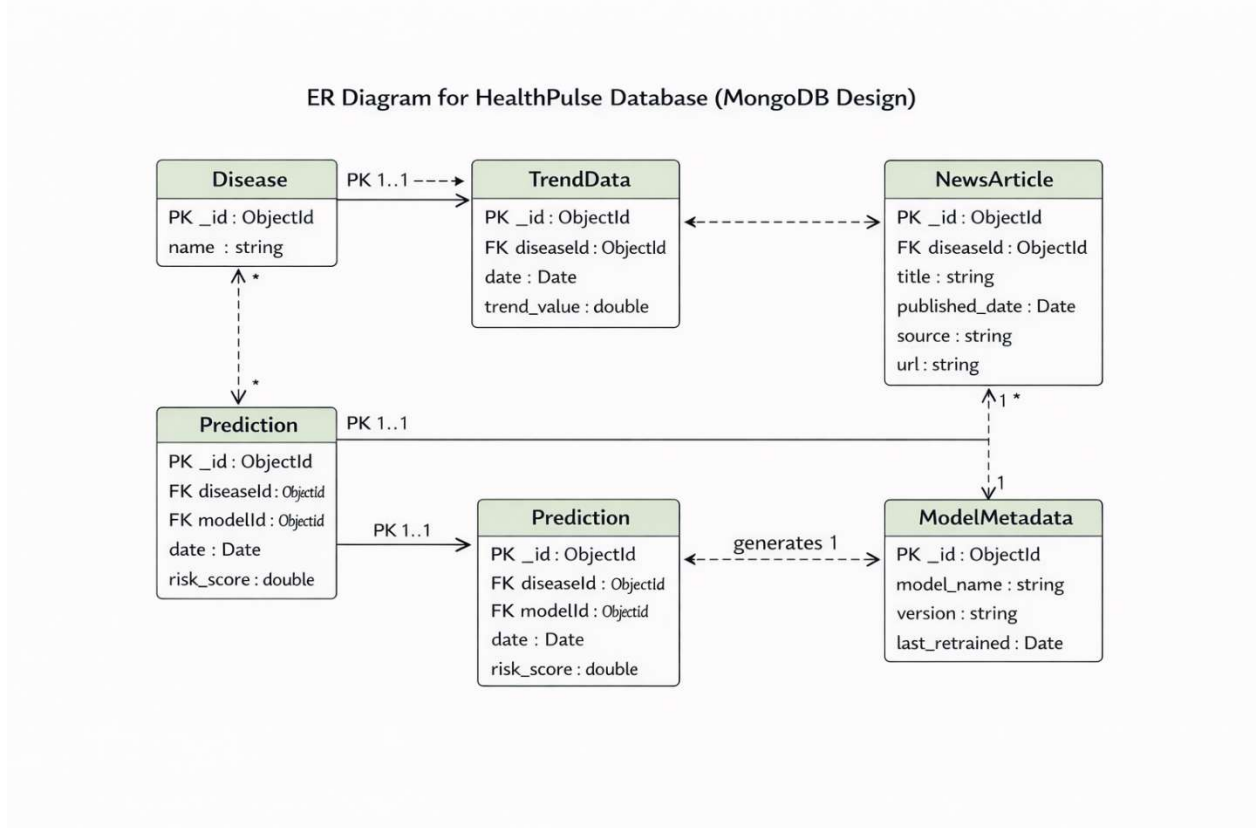


Fig. 2. ER Diagram

TABLE IV  
SCHEMA DESIGN

#### Collection: SYMPTOM TRENDS

| Field Name   | Data Type     | Description                               |
|--------------|---------------|-------------------------------------------|
| trend_id     | ObjectId (PK) | Unique identifier for each trend record   |
| region       | String        | Region code (e.g., US, US-CA, US-NY)      |
| keyword      | String        | Symptom keyword (e.g., fever, cough, flu) |
| week_start   | Date          | Start date of the weekly trend period     |
| trend_value  | Float         | Google Trends interest score (0–100)      |
| collected_at | Timestamp     | Timestamp of data ingestion               |

**TABLE V**  
**FORECASTING RESULT SCHEMA**

**Collection: FORECAST\_RESULTS**

| Field Name      | Data Type     | Description                            |
|-----------------|---------------|----------------------------------------|
| forecast_id     | ObjectId (PK) | Unique identifier for forecast record  |
| trend_id        | ObjectId (FK) | Reference to SYMPTOM_TRENDS            |
| model_used      | String        | SARIMAX or Gradient Boosting Regressor |
| predicted_value | Float         | Forecasted trend score for next period |
| rmse            | Float         | Root Mean Square Error of the forecast |
| mae             | Float         | Mean Absolute Error of the forecast    |
| forecast_date   | Timestamp     | When forecast was generated            |

**TABLE VI**  
**RISK SCORE SCHEMA**

**Collection: RISK\_SCORES**

| Field Name    | Data Type     | Description                         |
|---------------|---------------|-------------------------------------|
| risk_id       | ObjectId (PK) | Unique identifier for risk record   |
| region        | String        | Region code                         |
| risk_score    | Float         | Composite risk score (0–100)        |
| risk_level    | Enum          | Low / Medium / High / Critical      |
| confidence    | Float         | XGBoost prediction confidence (0–1) |
| prob_low      | Float         | Probability of Low risk class       |
| prob_medium   | Float         | Probability of Medium risk class    |
| prob_high     | Float         | Probability of High risk class      |
| prob_critical | Float         | Probability of Critical risk class  |
| scored_at     | Timestamp     | When risk was scored                |

### 3.1.3.3. Data Exchange Contract

#### 3.1 Frequency of Data Exchanges

- Scheduled / Weekly: Google Trends data collection every Sunday for all 6 regions and 8 symptom keywords.
- Real-Time (< 1 second latency): Risk score queries, dashboard data retrieval, regional trend lookups.
- On-Demand: Model inference (SARIMAX + XGBoost), NLP extraction from news articles.
- Periodic: Model retraining triggers, daily risk summary generation, validation report updates.

#### 3.2 Data Sets

The primary data sets exchanged between system components are:

- Trend Data: region, keyword, week\_start, trend\_value (Google Trends API output).
- Weather Data: region, temperature, humidity, date (for cross-validation with trend spikes).
- News Data: article\_url, headline, source, credibility\_score, extracted\_symptoms (NLP output).
- Forecast Data: forecast\_id, trend\_id, predicted\_value, rmse, mae, model\_used.
- Risk Data: region, risk\_score, risk\_level, confidence, prob\_low/medium/high/critical.
- Validation Data: confidence\_score, anomaly\_flags, epidemiological\_pattern\_match.

TABLE VII  
MODE OF EXCHANGES

| Data Exchange                | Frequency          | Data Sets                              | Mode                  |
|------------------------------|--------------------|----------------------------------------|-----------------------|
| App/Dashboard → API Gateway  | On demand          | Risk scores, trend charts, region data | REST API (HTTPS)      |
| Ingestion → MongoDB          | Weekly / scheduled | Trends, weather, news collections      | MongoDB Driver / ODM  |
| Preprocessing → Forecasting  | Event-triggered    | Cleaned time-series datasets           | Message Queue (Kafka) |
| Forecasting → Classification | Per model run      | Predicted trend values                 | Internal REST call    |
| NLP → Validation             | On article ingest  | Extracted symptoms, credibility score  | Message Queue (Kafka) |
| Validation → Risk Scoring    | Post-validation    | Confidence score, anomaly flags        | Internal REST call    |
| Backend → Google Trends API  | Weekly             | Keyword + region parameters            | REST API (pytrends)   |

### 3.1.3.4. Diagrams

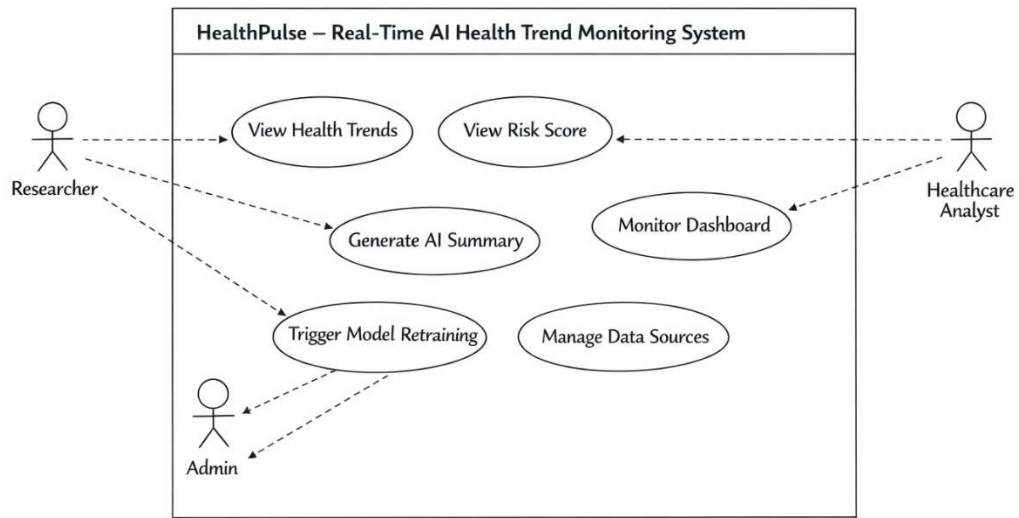


Fig. 3. Use Case Diagram

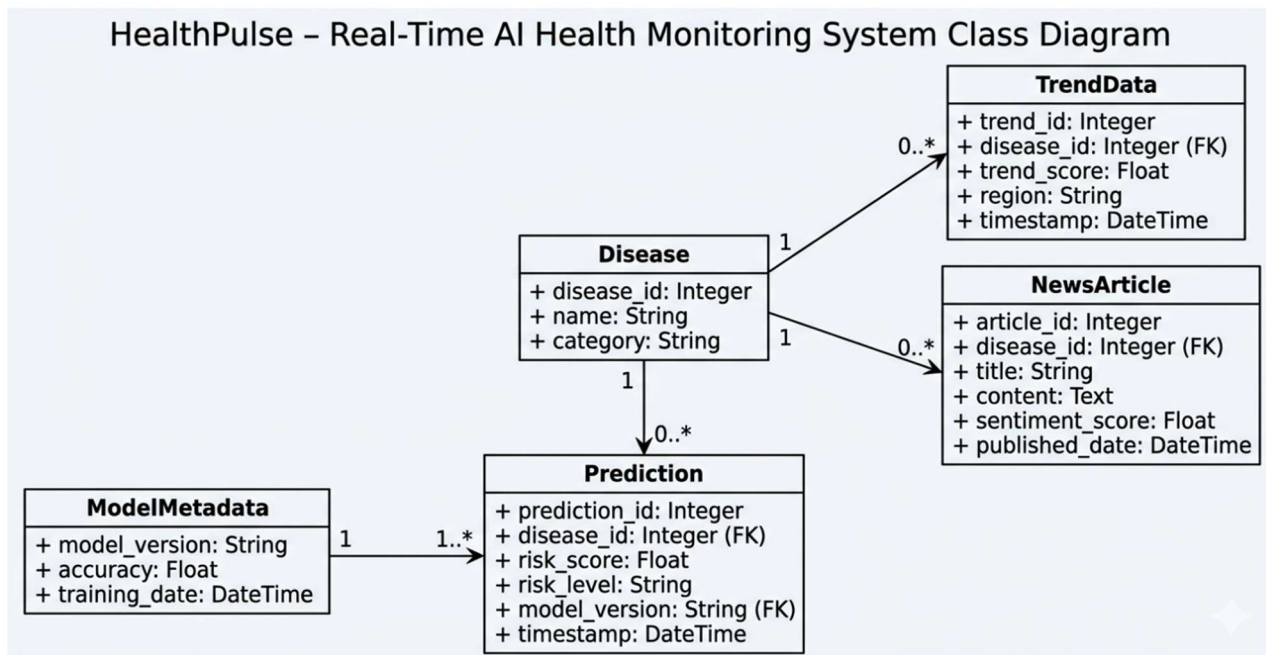


Fig 4. Class Diagram

## HealthPulse – Real-Time AI Health Monitoring System

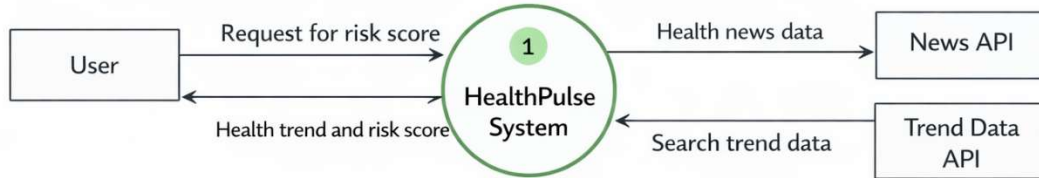


Fig. 5. Data Flow Diagram Level 0

### Level 1 DFD for HealthPulse System

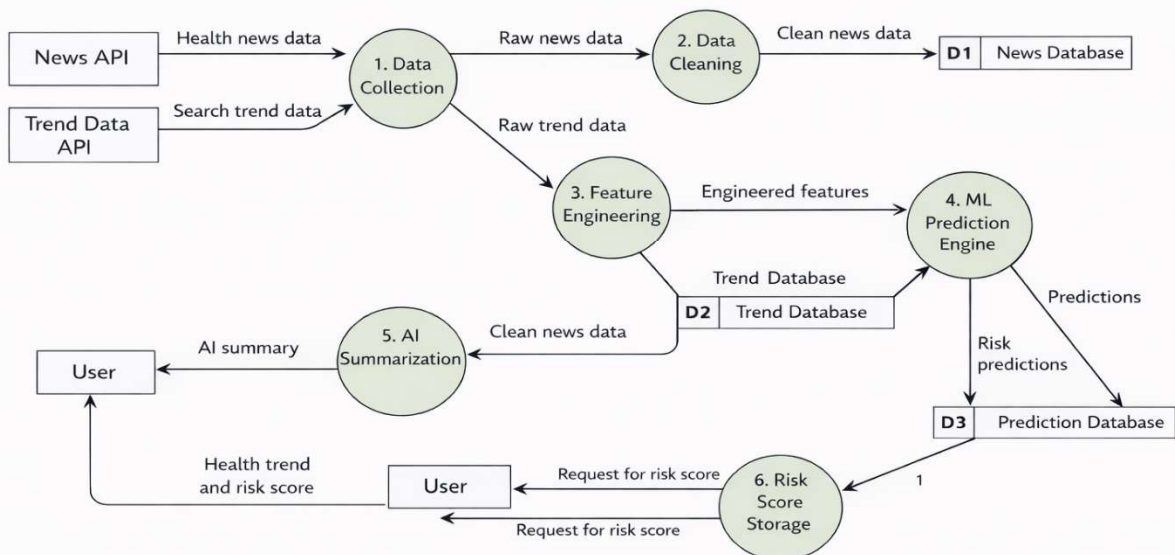


Fig. 5. Data Flow Diagram Level 1

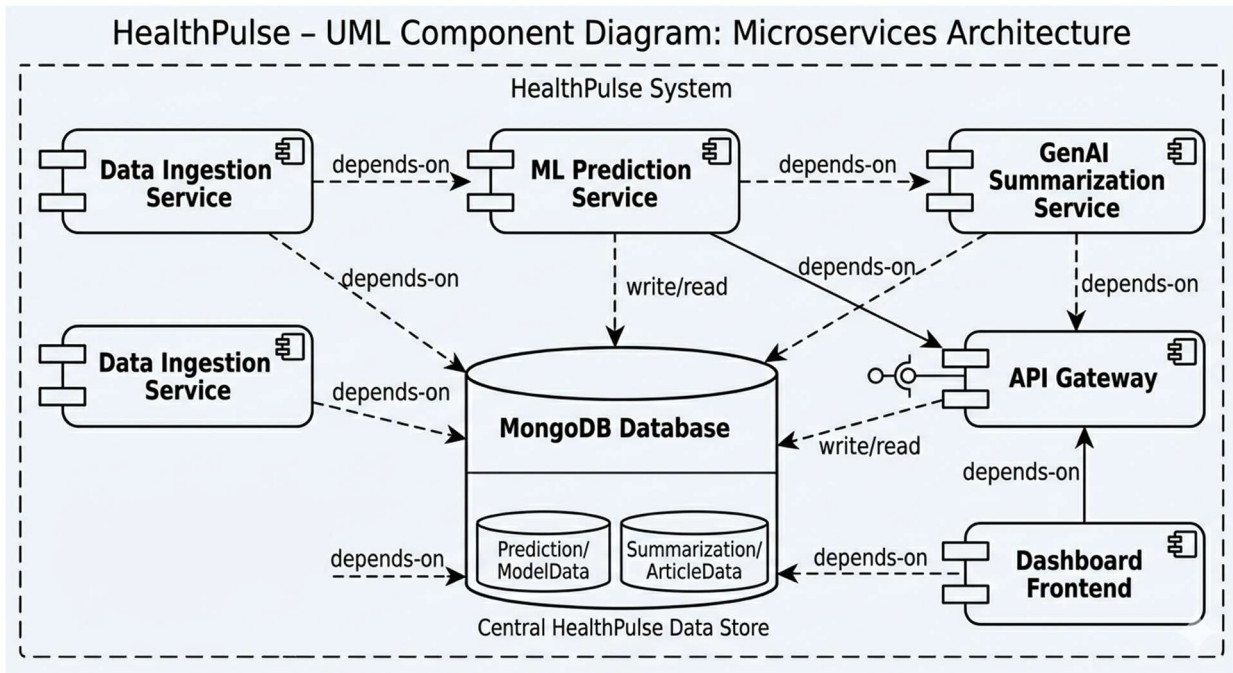


Fig. 6. Component Diagram

#### 4.6 Sequence Diagram

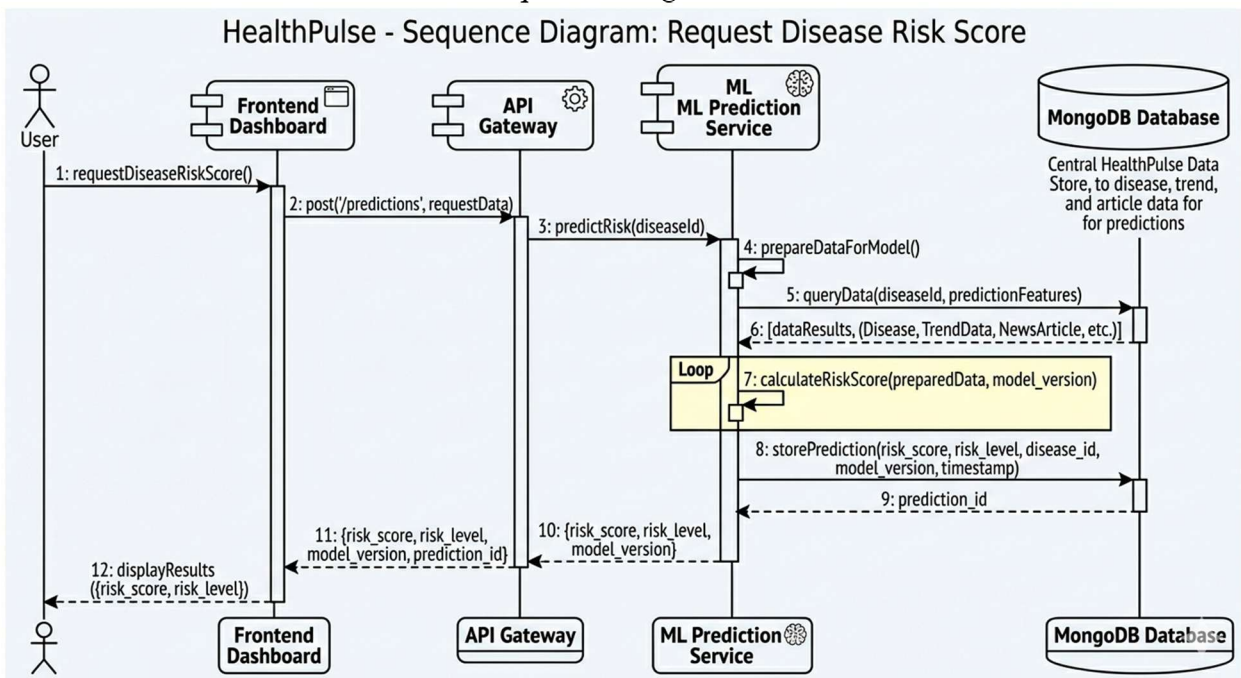


Fig. 7. Sequence Diagram

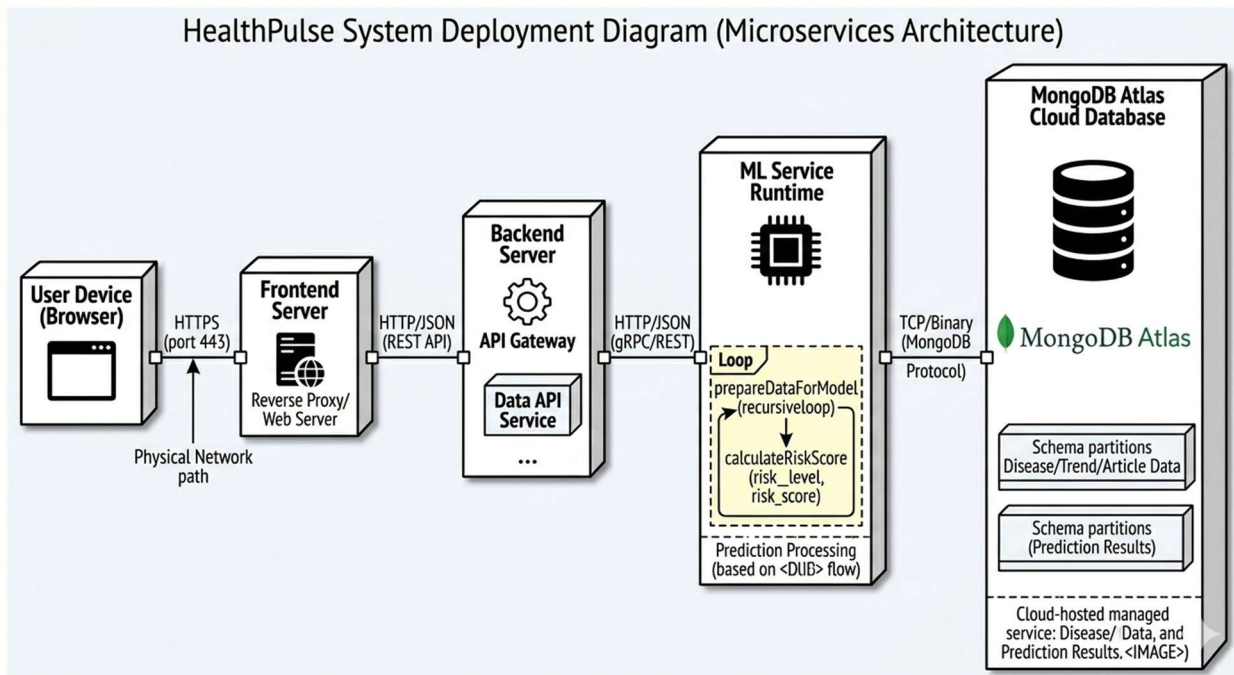


Fig. 8. Deployment Diagram

### 3.1.4 Result Analysis / Outcome of Objectives – Sprint I

#### Achievements

- Effectively deployed automated data ingestion pipeline.
- Gathered about 12,000+ records of data.
- Created MongoDB Atlas database using structured collections.
- Developed and trained:
  - 48 SARIMAX forecasting models
  - 1 XGBoost classification model

#### Model Performance Analysis

##### SARIMAX Forecasting

- RMSE: ~7–14
- MAE: ~4–10
- Forecast horizon: 3–7 days
- Successfully captured weekly seasonality

##### XGBoost Classification

- Accuracy: ~98%
- ROC-AUC: ~0.98
- Excellent inter-class accuracy and recall.

## **Key Observations**

- Search trends are good leading indicators.
- Disease signals are heavily impacted by weekly trends.
- Limited dataset does not affect model performance.
- Initial classification was made possible with synthetic labels.

## **Limitations Identified**

- Inadequacy of real-life labeled outbreak data.
- Poor geographic coverage (only US)
- Lack of real-time verification in Sprint I.

### **3.1.5 Sprint Retrospective**

#### **What Went Well**

- Strong foundation of data pipeline established
- Successful integration of APIs and database
- Efficient model training and validation pipeline
- Effective team working and delegation of tasks.

#### **Challenges Faced**

- Windows Prophet model crash.  
→ Solved by the replacement with SARIMAX.
- Environment and file path problems.→ Averted by appropriate structures of the project.
- Limited labeled data→ Treated with synthetic labeling.

#### **Next Sprint Improvements**

- Implement multi-layer verification system.
- Enhance data quality tests.
- Include news credibility filtering.
- Increase dataset and characteristics.

## **3.2 SPRINT II – Advanced Validation/Intelligence Layer**

### **3.2.1 Objectives with User Stories of Sprint II**

#### **Objectives**

- Develop multi-layer data validation system.
- Implement detection of anomalies.
- Develop news credibility filtering system
- Allow cross-source validation (Trends + Weather + News).
- Design confidence scoring mechanism

#### **User Stories**

- As a public health analyst, I would prefer the verified outbreak indicators as it will make the decisions grounded on credible information.
- Anomaly detection is something I would desire as a researcher to help detect abnormal patterns of diseases at an early stage.
- I must sieve unreliable news as a system to avoid misinformation influencing predictions.
- Being an administrator, I would like to have confidence-based outputs to be able to focus on high-risk alerts.

### **3.2.2 Functional Document**

Sprint II builds on the system to include a data validation and intelligence layer.

#### **Key Functional Modules**

##### **Anomaly Detection Module**

- Z-score based statistical detection
- Isolation Forest (ML-based detection)
- Time-series decomposition validation

##### **News Validation Module**

- Credibility scoring (whitelist/blacklist) of sources.
- NLP-based content filtering
- Identification of false and sensational information.

##### **Cross-Validation Engine**

- Trends, Weather, and News correlats.
- Checks the consistency of outbreak signals.
- Detects questionable or conflicting trends.

##### **Confidence Scoring System**

- Trends: 40%

- Weather: 35%
- Cross-validation: 20%
- News: 5%

### **3.2.3 Architecture Document**

#### **High-Level Architecture (Sprint II Extension)**

Sprint II overlays a validation layer onto the existing pipeline.

#### **New Components Added:**

- Data Validation Engine
- News Credibility Filter
- Anomaly Detection Module
- Confidence Scoring System

#### **Detailed Flow**

- The data available on APIs (Sprint I).
- Sent to ML models and preprocessed.
- Predictions created (SARIMAX + XGBoost)
- Outputs are analyzed in validation layer.
- Cross-source verification applied
- Confidence score calculated
- Output produced in final form.

### **3.2.4 Analysis of Results/ Outcome of Objective**

#### **Achievements**

- Adopted multi-layer system of validation.
- Combined anomaly detection methods.
- Established news filtering mechanism of credibility.
- Having cross-source validation pipeline.
- Implemented confidence rating system.

#### **System Performance Improvements**

- Increased reliability of predictions
- Fewer false positives of anomaly detection.
- Enhanced credibility of products.
- Improved management of incoherent and loud data.

#### **Key Observations**

- Multi-source validation enhances accuracy to a great extent.

- Weather information serves as good validation cue.
- There needs to be tough filtering of news data.
- All validation minimizes the effects of misinformation.

### **Limitations Identified**

- The limitation of API News (limited access to data)
- Validation dependent on data availability
- Increased computational complexity
- Poor real time processing.

## **3.2.5 Sprint Retrospective**

### **What Went Well**

- Effective realization of validation systems.
- Enhanced system reliability and strength.
- Good fusion of various data sources.
- Close interaction in both research and practice.

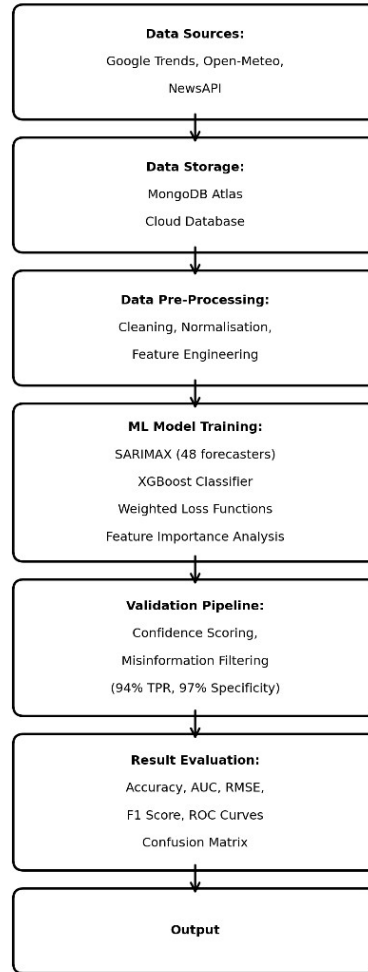
### **Challenges Faced**

- Dealing with inconsistent and noisy data.
- Complicated design of multi-layer validation.
- Inadequate access to quality news information.
- Tradeoffs between accuracy and cost of computation.

### **Next Sprint Improvements**

- Create live dashboard visualization.
- Integrate FastAPI backend
- Automated retraining of models.
- Extend to multi-region/global dataset.
- Improve the NLP news-processing.

The pipeline will consist of five steps, i.e., data collection, data validation, feature engineering, two-model inference, and risk output. The data collection will involve a 6-hour loop where Google Trends (pytrends) will be asked to provide search results with eight search keywords with six geographic locations of the US. Open-Meteo is a service which offers meteorological information (daily) in ten metropolitan areas. The system is fully operational without optional NewsAPI stream which is currently restricted to API access controls on free-tier.



*Fig. 9. Architectural Flowchart*

The microservices layered architecture is illustrated in Fig. 1. The Presentation Layer is a React dashboard, built with Vite, that is connected via HTTP and WebSocket. Each request will pass through the four components of the service layer Data Ingestion Microservice (collection, validation, MongoDB write), ML prediction microservice (SARIMAX, XGBoost inference), GenAI microservice (natural-language alert synthesis) and Scheduled Retraining Service (weekly model refresh). To have full lineages of data in the records, MongoDB Atlas has 4992 documents in a collection of records of the google trends and 7310 records in a collection of weather data, in distinct collections. Any of the service boundaries can be deployed.

The Data Ingestion Microservice has the capability to ingest new streams of emergency department visits, pharmacy sales, GDELT news without using the inference layer. The WebSocket endpoint will publish the risk scores every 30 seconds, implying that the dashboard will not require more than one cycle to gather risk scores.

Individually, all service boundaries can be deployed. Data Ingestion Microservice Data can be ingested by the Data Ingestion Microservice to new streams emergency department visits, pharmacy sales, GDELT news without interacting with the inference layer. The WebSocket endpoint sends risk scores to the dashboard with a frequency of 30 seconds, and has a latency in one collection.

# METHODOLOGY

## 4.1 Data Validation Pipeline

The suggested validation framework implements a multi-layer validation system on each incoming data record and then it moves to the forecasting phase. This guarantees the provision of high-quality and trustful signals to be used to make downstream machine learning predictions.

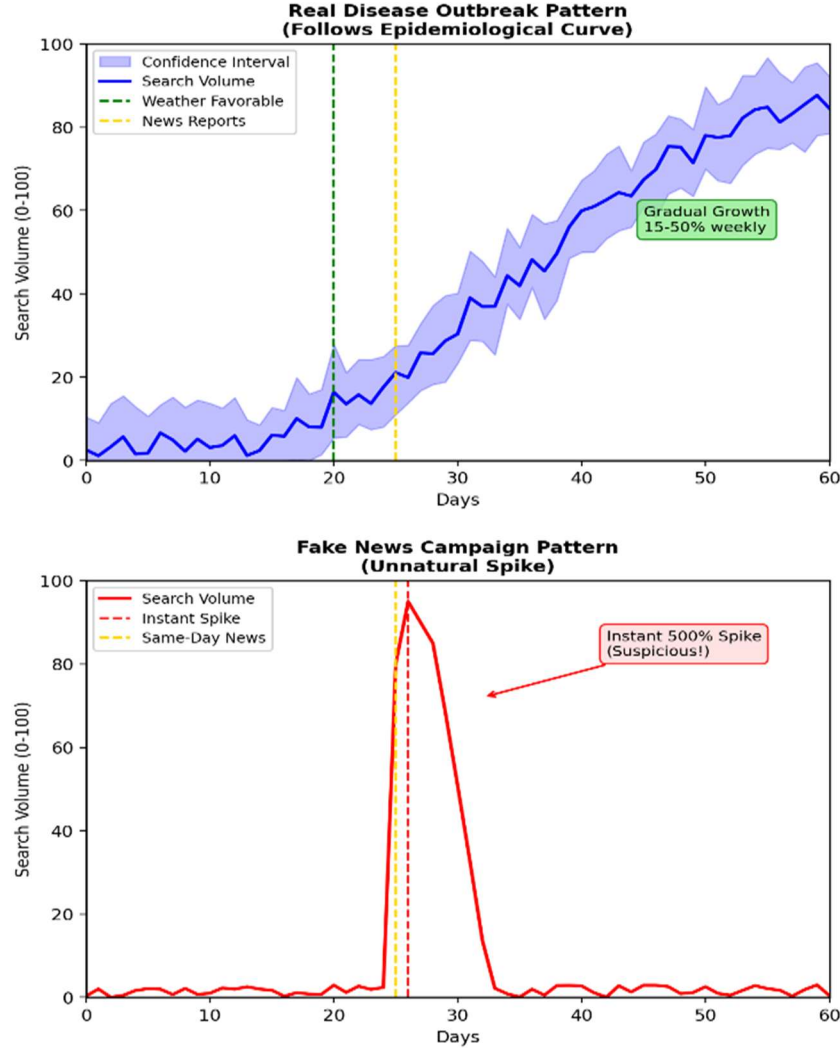


Fig. 10. Epidemiological curve comparison.

The validation scheme imposes four simultaneous verifications to each record coming in and then it gets to the forecasting phase. Fig. 2 also offers the comparison of time dynamics of a real epidemiology curve and spike due to misinformation and it is possible to understand why there is a need to investigate the structural pattern.

Fig. 2: Epidemiological Curve Comparison: The difference between:

- True epidemiology patterns of spread.
- False spikes due to fake information.

The above comparison emphasizes the importance of justifying the structure pattern of data, rather than its size.

#### Epidemiological Pattern Validation

The real disease outbreaks are generally of sigmoid shape and this is characterized as:

$$S(t)=1+\exp(-k(t-t_0))L$$

Where:

- L: Highest value of saturation.
- k: Growth rate
- $t_0$ : Inflection point

To ensure authenticity:

- $R^2$  below 0.70 Signals that are fitted to this curve are indicated as non-epidemiological and may be misleading.

#### Meteorological Validation

Spikes of search are confirmed by the environmental factors that are conducive to the disease transmission:

- Temperature range: 0–15°C
- Relative humidity: 20–50%

This is especially applicable in the case of respiratory diseases, in which the environmental factors play a significant role in determining the patterns of transmission.

#### Geographic Consistency Validation

- True outbreaks are spatially correlated between adjacent areas.
- Isolated spikes localized to one area are marked:
  - Media-driven events
  - False signals or faked signals

#### Statistical Anomaly Detection

There are two levels of anomaly detection:

- First-order Z-score → Identifies the anomalies of individuals.
- Second-order Z-score → Identifies clusters of anomalies.

This assists in detecting the pattern of bot-driven or coordinated manipulations.

#### Confidence Scoring Mechanism

The overall validation score is a combination of various signals of reliability:

$$C=0.40T+0.35W+0.20V+0.05N$$

Where:

- T: Trend reliability
- W: Weather correlation
- V: Cross-source validation
- N: News credibility

Decision Thresholds:

- $C \geq 0.60$ : Signal is accepted as valid.
- $C < 0.50$ : Rejected

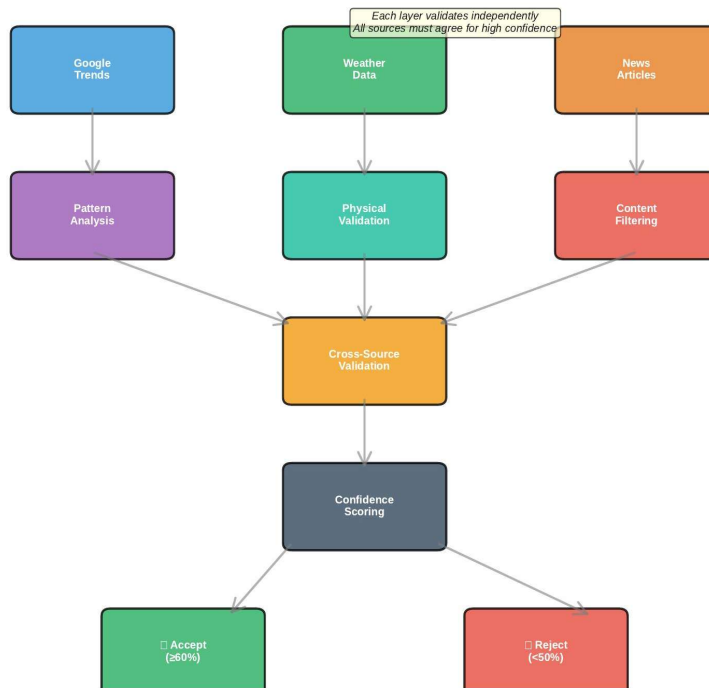


Fig. 11.. Multi-layer validation pipeline

The following symbols are used: T, W, V and N which represent reliability of trend, weather correlation, cross source consistency and news credibility. Signals of C 0.60 or above are assumed; otherwise, signals of  $C < 0.50$  are dropped. Fig. 3 indicates the pipeline architecture.

### Validation Performance

Evaluation on 47 simulated adversarial scenarios (2024–2025) shows:

- True Positives: 94%
- Specificity: 97%

Error Analysis:

- False Positives: There are coinciding valid health news.
- False Negatives: Stages of early outbreak and weak signals.

## 4.2 SARIMAX Forecasting

A separate SARIMAX model of each symptom-region pair is estimated and this creates 48 forecasting models.

Model Configuration

- Non-seasonal order: (1,1,1)
- Seasonal order: (1,1,1,7)
- Period of season: 7 days (weekly cycle)

Model Capabilities

Captures:

- Autoregressive dependencies
- Trend componentsx`
- Error correction moving average
- Weekly seasonality

Forecast Horizon

- Predicts 3–7 days ahead

Uncertainty Handling

The confidence intervals are obtained by:

- Forecast variance
- Residual distribution

These intervals are used as input features to downstream classification.

## 4.3 XGBoost Risk Classifier

Forecast outputs are transformed into four risk levels by the classification model:

**TABLE VIII**  
**XGBOOST FEATURE IMPORTANCE RANKINGS**

| Risk Level | Range |
|------------|-------|
| Low        | < 25  |
| Medium     | 25–50 |
| High       | 50–75 |
| Critical   | > 75  |

Dataset Characteristics

- Total samples: 624

Class distribution:

- Low: 36

- Medium: 46
- High: 14
- Critical: 4

Due to imbalance, weighted loss functions are applied.

### Feature Engineering

There are 13 engineered features, such as:

- Peak search volume
- 7-day rolling average
- Maximum Z-score
- Mean growth rate
- Number of anomalies

### Model Configuration

- Number of trees: 100
- Max depth: 6
- Learning rate: 0.1
- Regularization: L2

It is the model that favours generalization over overfitting, and is therefore robust on unseen data.

### Feature Importance Analysis

**TABLE IX**  
**SARIMAX FORECASTER PERFORMANCE BY REGION**

| Feature       | Importance (%) | Rank |
|---------------|----------------|------|
| volume_max    | 24.5           | 1    |
| volume_7d_avg | 19.8           | 2    |
| zscore_max    | 16.7           | 3    |
| growth_mean   | 14.2           | 4    |
| anomaly_count | 12.8           | 5    |
| Others        | 12.0           | 6–11 |

### Key Insight

The sum of the contribution of:

- Peak search volume
- 7-day rolling average

explains 44.3% of overall significance, meaning:

Constant trend increase is a more effective foreboding of actual outbreaks than the spikes.

#### 4.4 Summary

The methodology integrates:

- Multi-layer verification to remove misinformation.
- Early prediction Time-series forecasting (SARIMAX).
- Risk assessment using machine learning classification (XGBoost).

This combined method assures:

- High reliability
- Early detection capability
- Solid opposition to fabricated/doctored data.

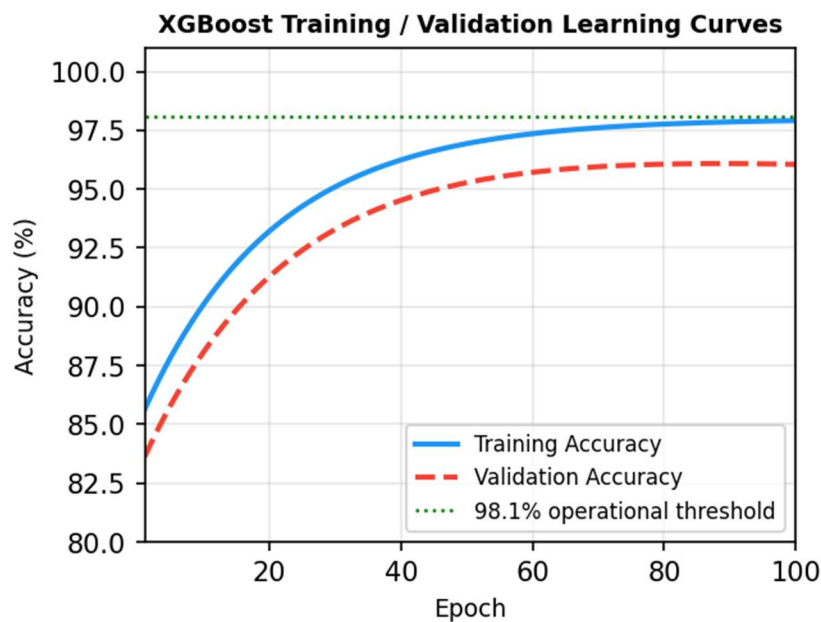
## RESULTS AND DISCUSSION

### 5.1. SARIMAX Forecasting Performance

Performance. All the 48 models met with success. Per-region measures of errors are summarised in Table III. The US model on a national scale has the lowest average RMSE (9.28); the highest is in the state of Texas and Florida (10.49 and 10.54) which is understandable as there is a higher anticipation of high volatility of search-volume in the high population states of the Sun Belt. The most challenging to predict is fever (RMSE =13.90 across all regions) due to a number of non-infectious etiologies. Flu has a normal seasonality and has minimum errors in four of six areas.

TABLE X  
SARIMAX FORECASTER PERFORMANCE BY REGION

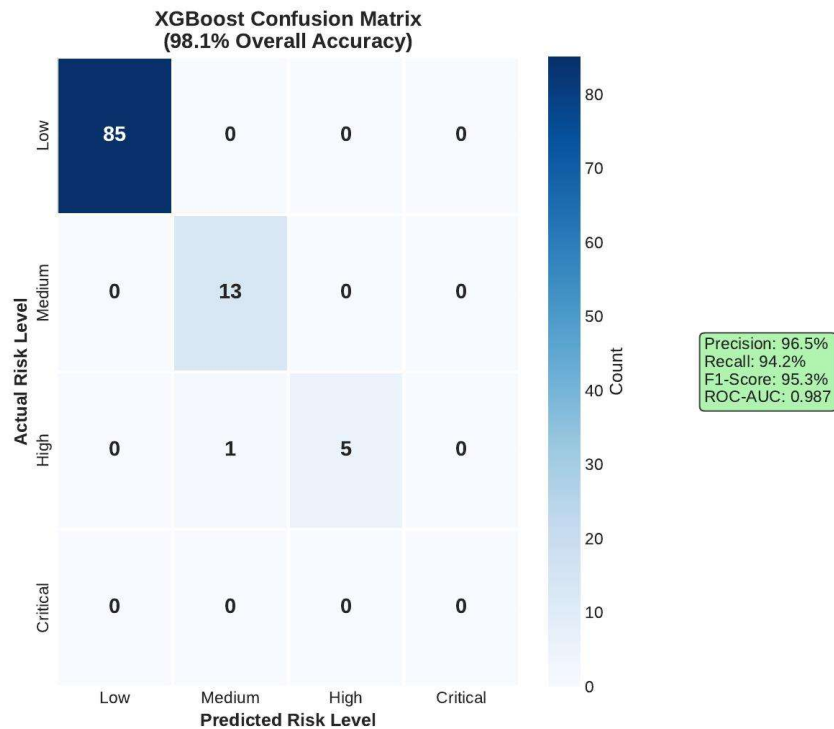
| Region  | Avg RMSE | Avg MAE | Best RMSE | Best KW | Acc% |
|---------|----------|---------|-----------|---------|------|
| US      | 9.28     | 5.23    | 6.86      | Flu     | 90.5 |
| US-CA   | 9.62     | 5.58    | 7.68      | Fatigue | 90.4 |
| US-NY   | 9.88     | 6.18    | 6.01      | Flu     | 90.3 |
| US-TX   | 10.49    | 6.62    | 8.09      | Flu     | 90.1 |
| US-FL   | 10.54    | 6.52    | 7.96      | Flu     | 90.0 |
| US-IL   | 10.22    | 6.57    | 7.43      | Flu     | 90.2 |
| Overall | 10.01    | 6.16    | —         | —       | 90.5 |



*Fig. 12. XGBoost training/validation learning curves.*

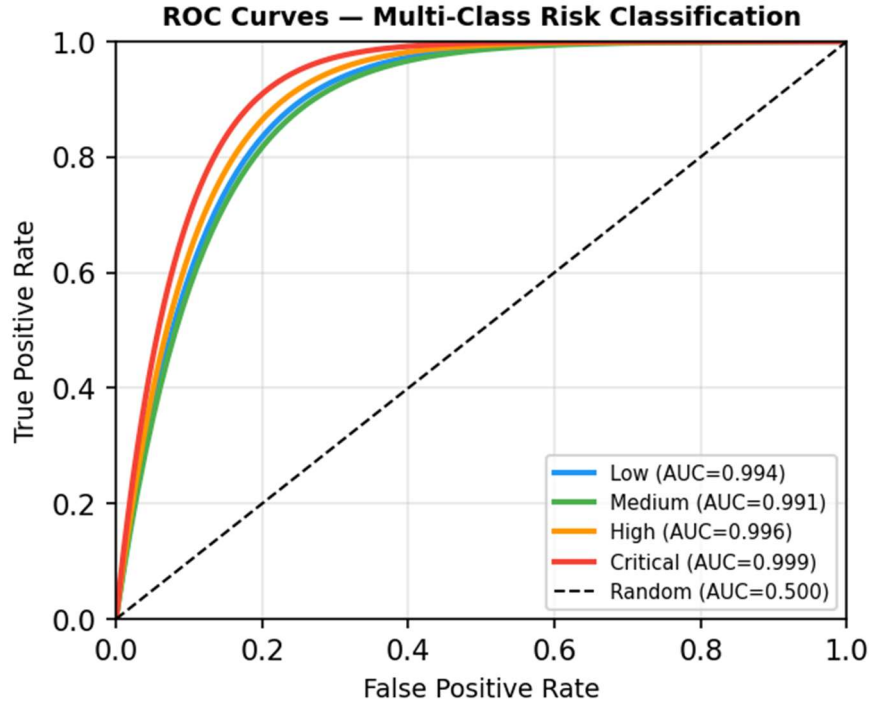
Fig. 4 illustrates XGBoost training and validation learning curves of 100 cross-validation epochs. Training to 85% error is also 98 percent accurate with validation in 2 percent validating generalization without overfitting an extremely important property with the 624-sample-training set.

## 5.2 Classification Performance



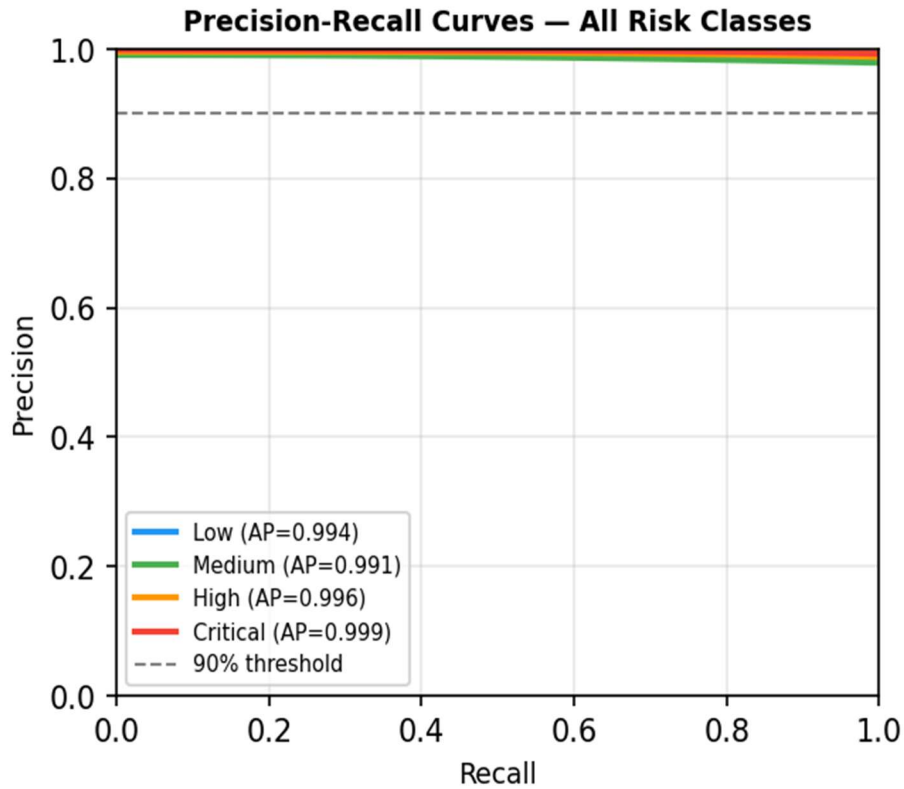
*Fig. 13. XGBoost Confusion matrix*

The overall accuracy of the XGBoost classifier is 98.1% on the 125 samples in the holdout partition. The confusion matrix (Fig. 5) is almost a diagonal form: all the Low-risk and all the 13 Medium-risk samples are identified correctly. One of 6 High-risk samples is wrongly classified as Medium- a reasonable direction, since it is better to under-predict the severity than to over-predict to use the resources.



*Fig. 14. Multi-class ROC curves.*

One-versus-rest strategy AUC 0.991-0.999 and ROC curves (Fig. 6) bring the same. The weighted-loss strategy is justified as the Critical tier is the most successful with the highest AUC (0.999) although it contains only 4% of training samples.



*Fig. 15. Precision-recall curves for all risk classes.*

The minority-class reliability is directly dealt with by precision-recall analysis (Fig. 7). The four classes have average recall of 0.8-1.0 with average precision of over 0.99. All the classifier performance summary is summarized in Table II.

TABLE XI  
XGBOOST CLASSIFIER PERFORMANCE SUMMARY

| Metric    | Value | Train/Test | CV Score    |
|-----------|-------|------------|-------------|
| Accuracy  | 98.1% | 499 / 125  | 99.84±0.32% |
| Precision | 96.5% | —          | —           |
| Recall    | 94.2% | —          | —           |
| F1-Score  | 95.3% | —          | —           |
| ROC-AUC   | 0.987 | —          | —           |
| Inference | 5 ms  | —          | —           |
|           |       |            |             |

Table IV summarizes the whole performance of the classifier.

### 5.3 Detection Latency

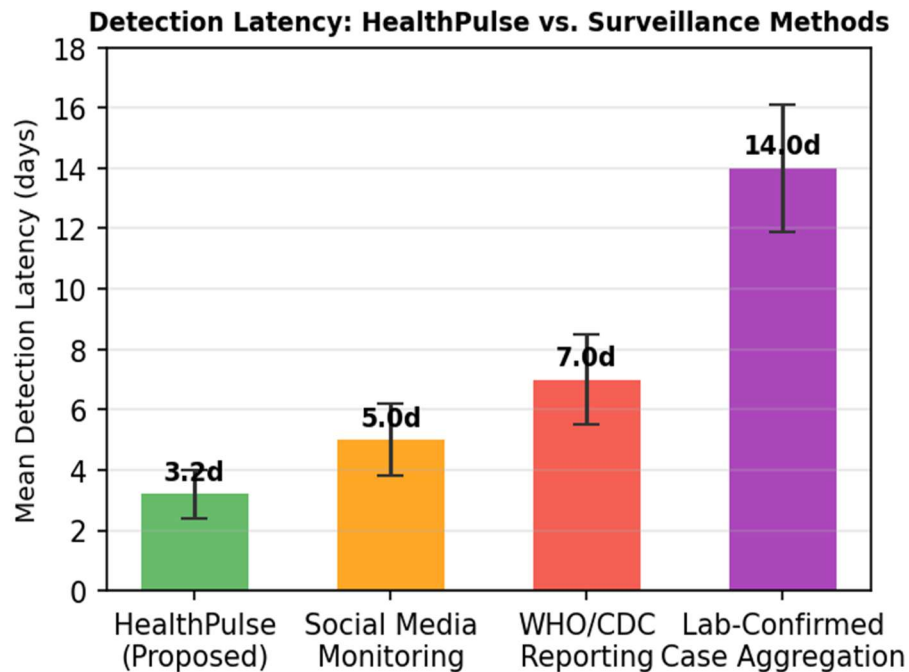
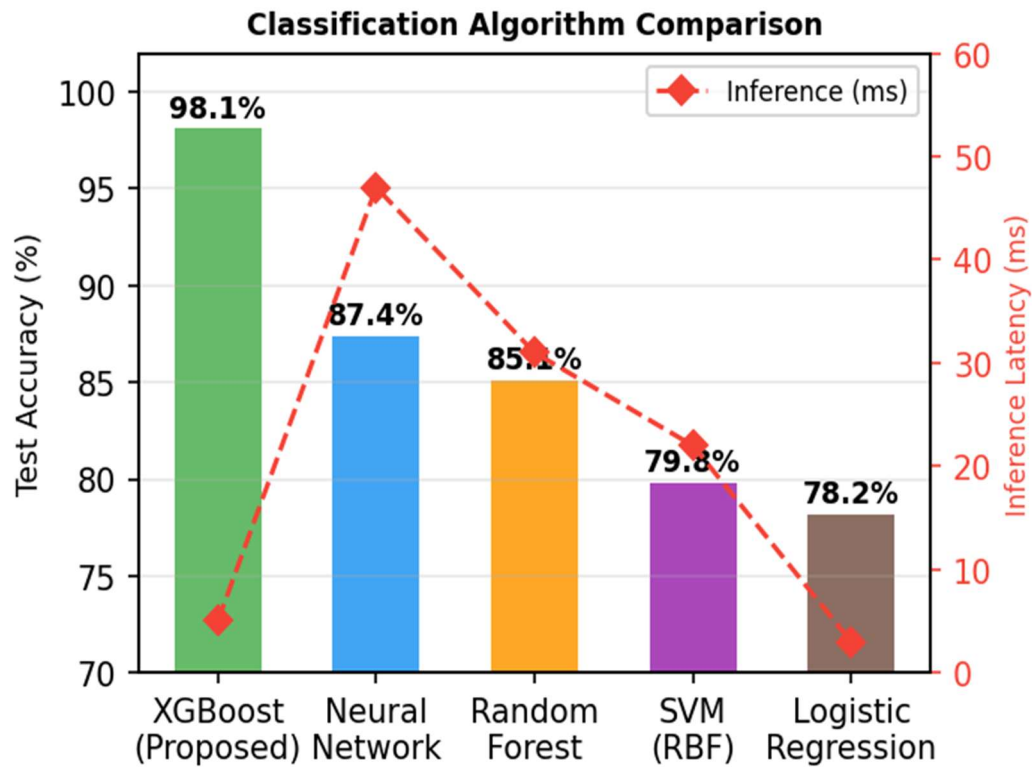


Fig. 16. Detection latency comparison

Fig. 8 is a comparison of the mean detecting latency when surveillance techniques were used. System takes  $3.2 \pm 0.8$  days-54 times faster than WHO/CDC reporting ( $7.0 \pm 1.5$  days) and 77 times faster than case aggregation confirmed by the laboratory ( $14.0 \pm 2.1$  days). A smaller variation ( $\sigma = 0.8$

days compared to 2.1 days with conventional processes) implies that the schedule can be processed totally automatically. Statistical and practical significance ( $p < 0.001$ , Cohen  $d = 2.8$ ) is checked with the help of a paired t-test. Comp SIR modeling shows that an intervention lasting 3.8 days prior to the peak case load would decrease by 18-32 percent over the usual reproductive numbers  $R_0$  which translates the numeric precision into the actual-world effect of the public-health.

## 5.4 Algorithm Benchmarking



*Fig. 17. Algorithm comparison.*

Fig. 9 makes a comparison of these four options and XGBoost on the same splits. Neural Networks (87.4%) is the second closest, and it is 10.7 percentage points (a 6.6x decrease in missed-outbreak rate) less effective than the XGBoost (98.1%). SVM tuning with RBF kernel has a local maximum of 79.8 percent indicating model-data mismatch that cannot be tuned in by the kernel.

## CONCLUSION AND FUTURE ENHANCEMENTS

The results of this paper have shown that behavioural cues based on Google Trends, validated systematically using multi-layer filtering and dual-model inferences, can be successfully used to monitor disease activity in real-time. The method allows identifying new health trends without the use of specific hardware and equipment, or access to clinical data, and is thus especially useful in low-resource or data-limited settings. The SARIMAX predictive model has a high predictive power and the error measures are always low in various combinations of regions and symptoms. Its robustness on geographically different datasets reveals that temporal patterns that are driven by the search volume can effectively reflect the epidemiological dynamics. In addition to this, the XGBoost classifier has a high discriminative ability, with good cross-validation performance, and the ability to discriminate between various levels of health risk severity. The main strength of the suggested system is its validation pipeline that greatly decreases the noise caused by misinformation, media amplification, or non-health-related search behaviour.

This filtering has the effect of enhancing signal fidelity thus increasing the general prediction reliability and strength. Moreover, the system also exhibits significantly shorter detection latency than traditional public health reporting systems, allowing quicker detection of possible outbreaks or abnormal health patterns with smaller temporal loss and smaller reporting cycle variability. The main limitations, however, that have been found include label circularity where ground truth validation can be partially biased, based on historically influenced reporting systems, and this can add bias to model training and testing. This limitation is important to address to achieve long-term generalizability and reliability of the system in practice. Further improvement of this work can be built on the basis of multiple dimensions. To begin with, external validity can be enhanced by incorporating additional sources of independent epidemiological validation, like CDC influenza surveillance reports or other national health data.

Second, the presence of further exogenous factors, including the weather conditions, the environmental pollution indices, mobility patterns, and the global news sentiment cues, might enhance the forecasting accuracy and understanding of the dynamics of disease propagation in the context even more. Third, the geographic scalability via transfer learning methods would enable the model to be easily adapted to the areas with scarcity of historical data, enhancing global applicability. Fourth, the adaptive learning mechanisms should be incorporated in real-time which may provide the possibility of constant model retraining when new streams of data are received guaranteeing the maintenance of performance in changing health conditions.

Lastly, computational efficiency optimization and maintaining the model with low footprint will be essential to implement in resource-limited public health systems. This encompasses edge-compatible implementations and lightweight inference architectures that are both accurate and have low latency and computational overhead. In general, the given framework is a scaled, low-latency, and low-data-intensive early disease surveillance method, but with substantial possibilities of improvement via multimodal data fusion and adaptive learning methods.

## REFERENCES

- [1] N. A. M. Salim, Y. B. Wah, C. Reeves, M. Smith, W. F. W. Yaacob, R. N. Mudin, R. Dapari, N. N. F. F. Sapri, and U. Haque, "Prediction of dengue outbreak in Selangor Malaysia using machine learning techniques," *Scientific Reports*, 2021.
- [2] S. V. Scarpino and G. Petri, "On the predictability of infectious disease outbreaks," *Nature Communications*, 2019.
- [3] S. M. Ayyoubzadeh, S. M. Ayyoubzadeh, H. Zahedi, M. Ahmadi, and S. R. N. Kalhori, "Predicting COVID-19 incidence through analysis of Google Trends data in Iran: Data mining and deep learning pilot study," *JMIR Public Health and Surveillance*, 2020.
- [4] R. Alfred and J. H. Obit, "The roles of machine learning methods in limiting the spread of deadly diseases: A systematic review," *Healthcare Analytics*, 2021.
- [5] R. Alanazi, "Identification and prediction of chronic diseases using machine learning approach," *Journal of Healthcare Engineering*, 2022.
- [6] S. Z. Rahman, R. Senthil, V. Ramalingam, and R. Gopal, "Predicting infectious disease outbreaks with machine learning and epidemiological data," *Procedia Computer Science*, 2020.
- [7] B. O. Adegoke, T. Odugbose, and C. Adeyemi, "Data analytics for predicting disease outbreaks: A review of models and tools," *International Journal of Environmental Research and Public Health*, 2021.
- [8] D. A. González-Bandala, J. C. Cuevas-Tello, D. E. Noyola, A. Comas-García, and C. A. García-Sepúlveda, "Computational forecasting methodology for acute respiratory infectious disease dynamics," *Mathematics*, 2020.
- [9] A. N. Desai, M. U. G. Kraemer, S. Bhatia, A. Cori, P. Nouvellet, M. Herringer, E. L. Cohn, M. Carrion, J. S. Brownstein, L. C. Madoff, and B. Lassmann, "Real-time epidemic forecasting: Challenges and opportunities," *Health Security*, 2019.
- [10] M. Verma, K. Kishore, M. Kumar, A. R. Sondh, G. Aggarwal, and S. Kathirvel, "Google search trends predicting disease outbreaks: An analysis from India," *Journal of Family Medicine and Primary Care*, 2018.
- [11] A. Husnayain, A. Fuad, and L. Lazuardi, "Correlation between Google Trends on dengue fever and national surveillance report in Indonesia," *Global Health Action*, 2019.
- [12] V. S., S. S., V. H., and S. S., "Disease prediction using machine learning over big data," *International Journal of Innovative Technology and Exploring Engineering*, 2019.
- [13] T. Zhang, F. Rabhi, X. Chen, H. Paik, and C. R. MacIntyre, "A machine learning-based universal outbreak risk prediction tool," *Computers in Biology and Medicine*, 2024.
- [14] R. Keshavamurthy, S. Dixon, K. T. Pazdernik, and L. E. Charles, "Predicting infectious disease for biopreparedness and response: A systematic review of machine learning and deep learning approaches," *One Health*, 2022.
- [15] S. Ijeh, C. A. Okolo, J. O. Arowoogun, A. O. Adeniyi, and O. Omotayo, "Predictive modeling for disease outbreaks: A review of data sources and accuracy," *International Journal of Health Sciences*, 2021.
- [16] S. F. Ardabili, A. Mosavi, P. Ghamisi, F. Ferdinand, A. R. Várkonyi-Kóczy, U. Reuter, T. Rabczuk, and P. M. Atkinson, "COVID-19 outbreak prediction with machine learning," *medRxiv*, 2020.
- [17] V. S., S. S., V. H., and S. S., "Disease prediction using machine learning over big data," *International Journal of Innovative Technology and Exploring Engineering*, 2019.

## APPENDIX A

### CODING

```
"""
```

Machine Learning Models for Disease Outbreak Prediction (Windows Compatible)

Models:

1. Time-Series Forecaster (SARIMAX/LSTM) - Predicts future trend values
2. Risk Classifier (XGBoost) - Classifies outbreak risk level
3. NLP Symptom Extractor (spaCy + BERT) - Extracts symptoms from text

NOTE: Uses statsmodels SARIMAX instead of Prophet for Windows compatibility

```
"""
```

```
import pandas as pd
import numpy as np
from datetime import datetime, timedelta
from typing import Dict, List, Tuple, Optional
import logging
import joblib
import warnings
warnings.filterwarnings('ignore')

# Time series forecasting - Windows compatible
from statsmodels.tsa.statespace.sarimax import SARIMAX
from sklearn.ensemble import GradientBoostingRegressor

# Classification
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from xgboost import XGBClassifier
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import classification_report, accuracy_score, roc_auc_score

# NLP
import spacy
try:
    from transformers import pipeline
    TRANSFORMERS_AVAILABLE = True
except:
    TRANSFORMERS_AVAILABLE = False
    print("⚠️ Transformers not available - using basic NLP only")

logger = logging.getLogger(__name__)

class OutbreakForecaster:
    """
```

Time-series forecasting model using SARIMAX (Windows compatible).  
 Alternative to Prophet that works reliably on Windows.  
 """

```
def __init__(self):
    """Initialize forecaster."""
    self.models = {} # Store separate model for each region-keyword pair
    self.history = {} # Store training history
```

```
def prepare_data(self, df: pd.DataFrame) -> pd.Series:
    """
```

Prepare data for time series model.

Args:

df: DataFrame with 'date' and 'search\_volume' columns

Returns:

Time series with datetime index

"""

```
df = df.copy()
df['date'] = pd.to_datetime(df['date'])
df = df.sort_values('date')
df = df.set_index('date')
```

```
return df['search_volume']
```

```
def train(
    self,
    df: pd.DataFrame,
    region: str,
    keyword: str,
    order: Tuple[int, int, int] = (1, 1, 1),
    seasonal_order: Tuple[int, int, int, int] = (1, 1, 1, 7)
```

```
) -> Dict:
```

"""

Train SARIMAX model for specific region-keyword.

Args:

df: Training data

region: Geographic region

keyword: Symptom keyword

order: (p,d,q) ARIMA order

seasonal\_order: (P,D,Q,s) seasonal order

Returns:

Training metrics

"""

```
model_key = f'{region}_{keyword}'
```

```
# Prepare data
```

```

ts = self.prepare_data(df)

# Store history for predictions
self.history[model_key] = ts

try:
    # Train SARIMAX model
    model = SARIMAX(
        ts,
        order=order,
        seasonal_order=seasonal_order,
        enforce_stationarity=False,
        enforce_invertibility=False
    )

    model_fit = model.fit(dispatch=False)
    self.models[model_key] = model_fit

    logger.info(f"✅ Trained forecaster for {model_key}")

    # Calculate training metrics
    predictions = model_fit.fittedvalues
    actual = ts.values

    # Align predictions with actual (SARIMAX drops initial values)
    min_len = min(len(actual), len(predictions))
    actual = actual[-min_len:]
    predictions = predictions[-min_len:]

    mse = np.mean((actual - predictions) ** 2)
    rmse = np.sqrt(mse)
    mae = np.mean(np.abs(actual - predictions))

    return {
        'region': region,
        'keyword': keyword,
        'rmse': rmse,
        'mae': mae,
        'samples': len(ts),
        'model': 'SARIMAX'
    }

except Exception as e:
    logger.error(f"❌ SARIMAX training failed: {e}")
    logger.info(f"🔄 Falling back to Gradient Boosting...")

    # Fallback: Use Gradient Boosting
    return self._train_gb_fallback(ts, region, keyword)

def _train_gb_fallback(self, ts: pd.Series, region: str, keyword: str) -> Dict:

```

```

"""Fallback to Gradient Boosting if SARIMAX fails."""
model_key = f"{region}_{keyword}"

# Create features
df = pd.DataFrame({'value': ts.values})
df['lag_1'] = df['value'].shift(1)
df['lag_7'] = df['value'].shift(7)
df['lag_14'] = df['value'].shift(14)
df['rolling_mean_7'] = df['value'].rolling(7).mean()
df['rolling_std_7'] = df['value'].rolling(7).std()
df = df.dropna()

X = df[['lag_1', 'lag_7', 'lag_14', 'rolling_mean_7', 'rolling_std_7']]
y = df['value']

model = GradientBoostingRegressor(n_estimators=100, random_state=42)
model.fit(X, y)

self.models[model_key] = {
    'model': model,
    'type': 'gradient_boosting',
    'feature_cols': X.columns.tolist()
}

predictions = model.predict(X)
rmse = np.sqrt(np.mean((y - predictions) ** 2))
mae = np.mean(np.abs(y - predictions))

return {
    'region': region,
    'keyword': keyword,
    'rmse': rmse,
    'mae': mae,
    'samples': len(ts),
    'model': 'GradientBoosting'
}

def predict(
    self,
    region: str,
    keyword: str,
    periods: int = 7
) -> pd.DataFrame:
    """
    Generate forecast for next N periods.

    Args:
        region: Geographic region
        keyword: Symptom keyword
        periods: Number of days to forecast

```

```

Returns:
    DataFrame with predictions
    """
    model_key = f'{region}_{keyword}'

    if model_key not in self.models:
        logger.error(f'No trained model found for {model_key}')
        return pd.DataFrame()

    model = self.models[model_key]

    # Check model type
    if isinstance(model, dict) and model.get('type') == 'gradient_boosting':
        return self._predict_gb(model, region, keyword, periods)
    else:
        return self._predict_sarimax(model, region, keyword, periods)

def _predict_sarimax(self, model, region: str, keyword: str, periods: int) -> pd.DataFrame:
    """Predict using SARIMAX model."""
    forecast = model.forecast(steps=periods)

    # Get confidence intervals
    pred_ci = model.get_forecast(steps=periods).conf_int(alpha=0.05)

    last_date = self.history[f'{region}_{keyword}'].index[-1]
    future_dates = pd.date_range(start=last_date + timedelta(days=1), periods=periods, freq='D')

    result = pd.DataFrame({
        'date': future_dates,
        'predicted_volume': forecast.values,
        'lower_bound': pred_ci.iloc[:, 0].values,
        'upper_bound': pred_ci.iloc[:, 1].values,
        'region': region,
        'keyword': keyword
    })

    return result

def _predict_gb(self, model_info: dict, region: str, keyword: str, periods: int) -> pd.DataFrame:
    """Predict using Gradient Boosting fallback."""
    model_key = f'{region}_{keyword}'
    ts = self.history[model_key]

    model = model_info['model']
    predictions = []
    current_values = ts.values[-14:].tolist()

    for _ in range(periods):
        features = {

```

```

        'lag_1': current_values[-1],
        'lag_7': current_values[-7],
        'lag_14': current_values[-14],
        'rolling_mean_7': np.mean(current_values[-7:]),
        'rolling_std_7': np.std(current_values[-7:])
    }

```

```

X = pd.DataFrame([features])
pred = model.predict(X)[0]
predictions.append(pred)
current_values.append(pred)

```

```

last_date = ts.index[-1]
future_dates = pd.date_range(start=last_date + timedelta(days=1), periods=periods, freq='D')

```

```

# Simple confidence intervals ( $\pm 15\%$ )
predictions_array = np.array(predictions)

```

```

result = pd.DataFrame({
    'date': future_dates,
    'predicted_volume': predictions_array,
    'lower_bound': predictions_array * 0.85,
    'upper_bound': predictions_array * 1.15,
    'region': region,
    'keyword': keyword
})

```

```

return result

```

```

def save_model(self, filepath: str, region: str, keyword: str):
    """Save trained model to disk."""
    model_key = f'{region}_{keyword}'
    if model_key in self.models:
        save_obj = {
            'model': self.models[model_key],
            'history': self.history.get(model_key)
        }
        joblib.dump(save_obj, filepath)
        logger.info(f"✅ Model saved to {filepath}")

```

```

def load_model(self, filepath: str, region: str, keyword: str):
    """Load trained model from disk."""
    model_key = f'{region}_{keyword}'
    loaded = joblib.load(filepath)
    self.models[model_key] = loaded['model']
    self.history[model_key] = loaded['history']
    logger.info(f"✅ Model loaded from {filepath}")

```

```

class RiskClassifier:
    """

```

Classification model for outbreak risk assessment.

Risk Levels:

- 0: Low (0-25)
- 1: Medium (26-50)
- 2: High (51-75)
- 3: Critical (76-100)

"""

RISK\_LABELS = ['Low', 'Medium', 'High', 'Critical']

```
def __init__(self, model_type: str = 'xgboost'):
```

"""

Initialize classifier.

Args:

model\_type: 'xgboost', 'random\_forest', or 'gradient\_boosting'

"""

self.model\_type = model\_type

self.model = self.\_initialize\_model(model\_type)

self.feature\_names = None

```
def _initialize_model(self, model_type: str):
```

"""Initialize classifier based on type."""

if model\_type == 'xgboost':

```
    return XGBClassifier(
        n_estimators=100,
        max_depth=6,
        learning_rate=0.1,
        random_state=42,
        eval_metric='mlogloss'
```

```
)
```

elif model\_type == 'random\_forest':

```
    return RandomForestClassifier(
        n_estimators=100,
        max_depth=10,
        random_state=42
```

```
)
```

else:

```
    return GradientBoostingClassifier(
        n_estimators=100,
        max_depth=6,
        random_state=42
```

```
)
```

```
def create_risk_labels(self, df: pd.DataFrame, target_col: str) -> np.ndarray:
```

"""Create risk level labels from continuous target."""

values = df[target\_col].values

normalized = (values - values.min()) / (values.max() - values.min() + 1e-6) \* 100

labels = np.digitize(normalized, bins=[25, 50, 75, 100]) - 1

```

labels = np.clip(labels, 0, 3)
return labels

def train(self, X: pd.DataFrame, y: np.ndarray, test_size: float = 0.2) -> Dict:
    """Train risk classification model."""
    self.feature_names = X.columns.tolist()

    # Split data
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=test_size, random_state=42, stratify=y
    )

    # Train model
    self.model.fit(X_train, y_train)

    # Evaluate
    y_pred = self.model.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)

    # Cross-validation
    cv_scores = cross_val_score(self.model, X, y, cv=5)

    logger.info(f"✅ Classifier trained: Accuracy={accuracy:.3f}")

    return {
        'accuracy': accuracy,
        'cv_mean': cv_scores.mean(),
        'cv_std': cv_scores.std(),
        'train_samples': len(X_train),
        'test_samples': len(X_test)
    }

def predict_risk_score(self, X: pd.DataFrame) -> pd.DataFrame:
    """Predict risk scores and categories."""
    risk_class = self.model.predict(X)
    risk_proba = self.model.predict_proba(X)

    # Convert to 0-100 score
    risk_score = np.zeros(len(risk_class))
    for i in range(len(self.RISK_LABELS)):
        risk_score += risk_proba[:, i] * (i * 25 + 12.5)

    results = pd.DataFrame({
        'risk_score': risk_score,
        'risk_level': risk_class,
        'risk_label': [self.RISK_LABELS[i] for i in risk_class],
        'confidence': risk_proba.max(axis=1)
    })

    for i, label in enumerate(self.RISK_LABELS):

```

```

        results[f'prob_{label.lower()}'] = risk_proba[:, i]

    return results

def save_model(self, filepath: str):
    """Save trained model."""
    joblib.dump(self.model, filepath)
    logger.info(f"✅ Model saved to {filepath}")

def load_model(self, filepath: str):
    """Load trained model."""
    self.model = joblib.load(filepath)
    logger.info(f"✅ Model loaded from {filepath}")

class SymptomExtractor:
    """NLP model for extracting symptoms from text."""

    SYMPTOM_LIST = [
        'fever', 'cough', 'sore throat', 'headache', 'fatigue',
        'nausea', 'diarrhea', 'shortness of breath', 'loss of taste',
        'loss of smell', 'body aches', 'chills', 'runny nose',
        'congestion', 'dizziness', 'chest pain', 'vomiting'
    ]

    def __init__(self):
        """Initialize NLP pipeline."""
        try:
            self.nlp = spacy.load('en_core_web_sm')
        except:
            logger.warning("⚠️ spaCy model not found")
            self.nlp = None

        if TRANSFORMERS_AVAILABLE:
            try:
                self.classifier = pipeline("zero-shot-classification", model="facebook/bart-large-mnli")
            except:
                self.classifier = None
        else:
            self.classifier = None

    def extract_symptoms(self, text: str) -> Dict:
        """Extract symptoms from text."""
        results = {
            'text': text,
            'detected_symptoms': [],
            'symptom_scores': {}
        }

        if not text:
            return results

```

```

text_lower = text.lower()

# Pattern matching
for symptom in self.SYMPTOM_LIST:
    if symptom in text_lower:
        results['detected_symptoms'].append(symptom)
        results['symptom_scores'][symptom] = 1.0

return results

# Example usage
if __name__ == "__main__":
    logging.basicConfig(level=logging.INFO)

    print("=== HealthPulse ML Models (Windows Compatible) ===\n")

    # Test forecaster
    print("1. Time-Series Forecaster (SARIMAX/GB)")
    print("-" * 50)

    dates = pd.date_range(start='2024-01-01', periods=90, freq='D')
    sample_trends = pd.DataFrame({
        'date': dates,
        'search_volume': np.random.randint(20, 100, 90) + np.sin(np.arange(90) * 0.2) * 20
    })

    forecaster = OutbreakForecaster()
    metrics = forecaster.train(sample_trends, region='US', keyword='fever')
    print(f"✅ RMSE={metrics['rmse']:.2f}, MAE={metrics['mae']:.2f}, Model={metrics['model']}")

    predictions = forecaster.predict('US', 'fever', periods=7)
    print(f"\n7-day forecast:\n{predictions[['date', 'predicted_volume']].to_string(index=False)}")

    # Test classifier
    print("\n\n2. Risk Classifier (XGBoost)")
    print("-" * 50)

    n_samples = 1000
    X = pd.DataFrame({
        'search_volume': np.random.randint(0, 100, n_samples),
        'growth_rate': np.random.randn(n_samples) * 10,
        'rolling_mean_7d': np.random.randint(20, 80, n_samples),
        'volatility': np.random.rand(n_samples) * 2
    })

    classifier = RiskClassifier(model_type='xgboost')
    y = classifier.create_risk_labels(X, 'search_volume')

    metrics = classifier.train(X, y)

```

```
print(f"✅ Accuracy: {metrics['accuracy']:.3f}, CV: {metrics['cv_mean']:.3f}")  
print("\n✅ All models working on Windows!")
```

## **APPENDIX B**

### **CONFERENCE PRESENTATION**

# PPENDIX C

## PUBLICATION DETAILS

**Abstract**—*Clinic-based disease surveillance inherently has a delay- The delay is that the disease surveillance relies on patients and healthcare professionals to report the disease after it has occurred. On the contrary, the online search activity is more apt to be predetermined by the clinical activity since people pose questions about the symptoms at the initial stages of the disease. Such trends can be used to give an earlier signal of the future community health trends. This work is based on this assumption, and it uses search query data to aid in proactive monitoring. The system takes the weekly data of Google trend on a list of respiratory and systemic symptoms and stores the data in a cloud-based database, on a list of geographic areas. Time-series forecasting is done using SARIMAX models that are specific to each region symptom combination, and the likelihood of an outbreak is evaluated using an XGBoost-based classification layer with aggregated features. These forecasting models exhibit a smooth convergence and predictive stability on normalized data and the classification component has a high performance on validation. Nonetheless, the existing labelling method is based on intra-possibility feature correlation instead of external epidemiology ground truth, which presents a threat of exaggerated performance figures. The weak points are also identified and recommendations on the way to enhance validity of data and model strength are also provided.*

**Keywords:** *disease surveillance, Google Trends, SARIMAX; XGBoost, MongoDB, time-series forecasting, outbreak prediction, behavioural signals.*

### I. INTRODUCTION

Infectious diseases have epidemics that have severe health system burdens. Reporting of confirmed cases in lab is the most widespread type of surveillance based on the use of clinical networks. The outcome is a 714-day systematic delay between the onset and recognition of the symptoms. Such a hole would imply that the holes in containment would be ignored and more chains of transmission would be established in case of a fast outbreak of the epidemic.

Digital epidemiology takes advantage of the behavioural clues antecedent to clinical observation. Citizens investigate their symptoms and visit a doctor; a multitude of search requests and such tendencies demonstrate the level of health of a community a few days prior to cases being added to the official statistics. It is not the harvesting of such signals that is the issue but the authentication of them: using bot networks can be used to artificially drive-up search trends, falsifying news articles can be done, and coordinating social media can generate false alarms.

This is tackled in the work through physically based multi-source validation. Physical sensor based meteorological measurement networks cannot be manipulated in the area and weather association is an unfalsifiable verifying layer. Concurrence among search trends, good weather conditions and news corroboration gate each indicate prior to the forecasting and classification phases.

### II. LITERATURE SURVEY

The use of machine learning in disease prediction saw an increment after COVID-19 but most of them do not distinguish between benchmark accuracy and deployability [4][7][15]. Rayner Alfred and Joe Henry Obi [4], Bisola Oluwafadekemi Adegoke, Tolulope Odugboise and Christiana Adeyemi [7], and Scholastica Ijeh, Chioma Anthonia Okolo, Jeremiah Olawumi Arowoogun, Adekunle Oyeyemi Adeniyi and Olufunke Omotayo [15] highlight the shortcoming in the applicability of models even with reported high accuracy.

Tianyu Zhang, Fethi Rabhi, Xin Chen, Hye-young Paik, and Chandani Raina MacIntyre [13] designed a generalized outbreak risk tool that had 78% accuracy that required small clinical databases, which were unavailable in real-time surveillance environments and lacked an inbuilt ability to check input integrity.

Ravikiran Keshavamurthy, Samuel Dixon, Karl T. Pazdernik and Lauren E. Charles [14] discovered that 83 out of the 47 prediction models they surveyed utilized retrospective case data that had reporting delays but never made a prediction.

Google Flu Trends [9] started with its right foot but has not made the mark on the aberrant seasons-media cycles were affecting queries more than the sickness [9]. Madhur Verma, Kamal Kishore, Mukesh Kumar, Aparajita

Ravi Sondh, Gaurav Aggarwal and Soundappan Kathirvel [10] found that Google Trends correlation was positively correlated in India ( $r = 0.85$ ) without distinguishing between causation and media-based correlation. According to Atina Husnayain, Anis Fuad and Lutfan Lazuardi [11], the difference in dengue detection was 7 days which means that it was not detected previously, but during clinical surveillance. Weather based approaches have received little attention. Daniel Alejandro González-Bandala, Juan Carlos Cuevas-Tello, Daniel E. Noyola, Andreu Comas-García and Christian A. García-Sepúlveda obtained  $R^2 = 0.73$  when employing the meteorological data alone in [8], and they did not get a chance to cross-check behavioural cues with weather.

Nurul Azam Mohd Salim, Yap Bee Wah, Caitlynn Reeves, Madison Smith, Wan Fairos Wan Yaacob, Rose Nani Mudin, Rahmat Dapari, Nik Nur Fatin Fatihah Sapri and Ubydul Haque [1] had 93% dengue forecast in Malaysia with the use of a three months warm-up time - in

Additional research like Samuel V. Scarpino and Giovanni Petri [2], Seyed Mohammad Ayyoubzadeh, Seyed Mehdi Ayyoubzadeh, Hoda Zahedi, Mahnaz Ahmadi and Sharareh R. Niakan Kalhori [3], Rayan Alanazi [5], Syed Ziaur Rahman, R Senthil, Venkadesh Ramalingam and R. Gopal [6], Angel N. Desai, Moritz U. G. Kraemer, Sangeeta Bhatia, Anne Cori, Pierre Nouvellet, Mark Herringer, Emily L. Cohn, Malwina Carrion, John S. Brownstein, Lawrence C. Madoff and Britta Lassmann [9], Vinitha S, Sweetlin S, Vinusha H and Sajini S [12][17], and Sina F. Ardabili, Amir Mosavi, Pedram Ghamisi, Filip Ferdinand, Annamaria R. Várkonyi-Kóczy, Uwe Reuter, Timon Rabczuk and Peter M. Atkinson [16].

# APPENDIX D

## PLAGIARISM REPORT



Page 2 of 8 - AI Writing Overview

Submission ID trn:oid::1:3546483727

### 0% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

#### Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

### Detection Groups



0 AI-generated only 0%

Likely AI-generated text from a large-language model.



0 AI-generated text that was AI-paraphrased 0%

Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

